گزارش پروژه: professional-document-scanner

پوشه: مسیر /home/behzad/.gemini/antigravity/scratch/professional-document-scanner

ساختار پوشه‌ها و فایل‌ها

```
app.js
background.js
composer.css
composer.html
composer.js
fonts/
  Vazirmatn-Bold.woff2
  Vazirmatn-Regular.woff2
icon-192.png
icon-512.png
icons/
  icon-128.png
  icon-16.png
  icon-32.png
  icon-48.png
index.html
jspdf.umd.min.js
jszip.min.js
manifest.json
processor.js
project_report.txt
styles.css
test-math.js
test-math2.js
test.js
```

محتوای فایل‌ها

فایل: app.js

```
document.addEventListener('DOMContentLoaded', () => {
    // UI Elements
    const dropZone = document.getElementById('drop-zone');
    const fileInput = document.getElementById('file-input');
    const workspace = document.getElementById('workspace');
    const thumbnailList = document.getElementById('thumbnail-list');
```

```
const canvasOriginal = document.getElementById('canvas-original');
const ctxOriginal = canvasOriginal.getContext('2d');
const canvasResult = document.getElementById('canvas-result');
const ctxResult = canvasResult.getContext('2d');
const resultContainer = document.getElementById('result-container');

const processBtn = document.getElementById('process-btn');
const downloadSingleBtn = document.getElementById('download-single-btn');
const downloadAllBtn = document.getElementById('download-all-btn');
const exportFormat = document.getElementById('export-format');
const addMoreBtn = document.getElementById('add-more-btn');

// Sliders & Controls
const controls = {
    mode: document.querySelectorAll('input[name="mode"]'),
    brightness: document.getElementById('brightness'),
    contrast: document.getElementById('contrast'),
    threshold: document.getElementById('threshold'),
    shadow: document.getElementById('shadow')
};

const thresholdContainer = document.getElementById('threshold-container');

// State
let images = []; // Array of { id, file, imgObj, processedDataUrl }
let currentImageIndex = -1;
let isProcessing = false;

// --- File Upload & Drag-Drop ---

// --- History & Undo/Redo ---
const undoBtn = document.getElementById('undo-btn');
const redoBtn = document.getElementById('redo-btn');
const resetImageBtn = document.getElementById('reset-image-btn');

function cloneCanvasOrImage(source) {
    const c = document.createElement('canvas');
    c.width = source.width || source.naturalWidth;
    c.height = source.height || source.naturalHeight;
    c.getContext('2d').drawImage(source, 0, 0);
    return c;
}

function saveHistoryState(imgObj) {
    if (!imgObj) return;
```

```
    if (imgObj.historyIndex < imgObj.history.length - 1) {
        imgObj.history = imgObj.history.slice(0, imgObj.historyIndex + 1);
    }

    // Keep max 15 states to avoid memory issues
    if (imgObj.history.length >= 15) {
        imgObj.history.shift();
        imgObj.historyIndex--;
    }

    imgObj.history.push({
        imgElement: cloneCanvasOrImage(imgObj.imgElement),
        eraserMask: cloneCanvasOrImage(imgObj.eraserMask),
        settings: JSON.parse(JSON.stringify(imgObj.settings))
    });
    imgObj.historyIndex++;
    updateHistoryButtons();
}

function updateUIFromSettings(settings) {
    if (!settings) return;

    ['brightness', 'contrast', 'threshold', 'shadow'].forEach(id => {
        const slider = document.getElementById(id);
        const valSpan = document.getElementById(`${id}-val`);
        if (slider && valSpan) {
            slider.value = settings[id];
            valSpan.textContent = settings[id];
        }
    });

    const modeRadios = document.querySelectorAll('input[name="mode"]');
    modeRadios.forEach(r => {
        r.checked = (r.value === settings.mode);
    });

    const thresholdContainer = document.getElementById('threshold-container');
    if (thresholdContainer) {
        thresholdContainer.style.opacity = settings.mode === 'bw' ? '1' : '0.5';
        thresholdContainer.style.pointerEvents = settings.mode === 'bw' ? 'auto' : 'none
    }
}

function updateHistoryButtons() {
    if (currentImageIndex === -1) {
```

```
        if(undoBtn) undoBtn.disabled = true;
        if(redoBtn) redoBtn.disabled = true;
        if(resetImageBtn) resetImageBtn.disabled = true;
        return;
    }

    const imgObj = images[currentImageIndex];
    if(undoBtn) undoBtn.disabled = imgObj.historyIndex <= 0;
    if(redoBtn) redoBtn.disabled = imgObj.historyIndex >= imgObj.history.length - 1;
    if(resetImageBtn) resetImageBtn.disabled = false;
}

if(undoBtn) {
    undoBtn.addEventListener('click', () => {
        if (currentImageIndex === -1) return;
        const imgObj = images[currentImageIndex];
        if (imgObj.historyIndex > 0) {
            imgObj.historyIndex--;
            applyHistoryState(imgObj);
        }
    });
}

if(redoBtn) {
    redoBtn.addEventListener('click', () => {
        if (currentImageIndex === -1) return;
        const imgObj = images[currentImageIndex];
        if (imgObj.historyIndex < imgObj.history.length - 1) {
            imgObj.historyIndex++;
            applyHistoryState(imgObj);
        }
    });
}

function applyHistoryState(imgObj) {
    const state = imgObj.history[imgObj.historyIndex];
    imgObj.imgElement = cloneCanvasOrImage(state.imgElement);
    imgObj.eraserMask = cloneCanvasOrImage(state.eraserMask);
    if (state.settings) {
        imgObj.settings = JSON.parse(JSON.stringify(state.settings));
        updateUIFromSettings(imgObj.settings);
    }
    updateHistoryButtons();
    updateOriginalPreview(); // Update the left side preview
    scheduleProcess(); // Update the right side processed result
}
```

```
if(resetImageBtn) {
    resetImageBtn.addEventListener('click', () => {
        if (currentImageIndex === -1) return;
        if (!confirm('                                                              .  '(( re

        const imgObj = images[currentImageIndex];
        const reader = new FileReader();
        reader.onload = (e) => {
            const img = new Image();
            img.onload = () => {
                imgObj.imgElement = img;
                imgObj.eraserMask = document.createElement('canvas');
                imgObj.eraserMask.width = img.naturalWidth;
                imgObj.eraserMask.height = img.naturalHeight;
                imgObj.history = [];
                imgObj.historyIndex = -1;

                const controls = {
                    brightness: document.getElementById('brightness'),
                    contrast: document.getElementById('contrast'),
                    threshold: document.getElementById('threshold'),
                    shadow: document.getElementById('shadow'),
                    mode: document.querySelectorAll('input[name="mode"]')
                };
                imgObj.settings = {
                    mode: Array.from(controls.mode).find(r => r.checked).value,
                    brightness: controls.brightness.value,
                    contrast: controls.contrast.value,
                    threshold: controls.threshold.value,
                    shadow: controls.shadow.value
                };

                saveHistoryState(imgObj);
                updateOriginalPreview();
                scheduleProcess();
            };
            img.src = e.target.result;
        };
        reader.readAsDataURL(imgObj.file);
    });
}

dropZone.addEventListener('click', () => fileInput.click());

dropZone.addEventListener('dragover', (e) => {
```

```
    e.preventDefault();
    dropZone.classList.add('dragover');
});

dropZone.addEventListener('dragleave', () => dropZone.classList.remove('dragover'));

dropZone.addEventListener('drop', (e) => {
    e.preventDefault();
    dropZone.classList.remove('dragover');
    handleFiles(e.dataTransfer.files);
});

fileInput.addEventListener('change', (e) => handleFiles(e.target.files));

addMoreBtn.addEventListener('click', () => fileInput.click());

function handleFiles(files) {
    if (!files.length) return;

    let fileProcessedCount = 0;

    Array.from(files).forEach(file => {
        if (!file.type.startsWith('image/')) return;

        const reader = new FileReader();
        reader.onload = (e) => {
            const img = new Image();
            img.onload = () => {
                const imgObj = {
                    id: Date.now() + Math.random().toString(36).substr(2, 9),
                    file: file,
                    name: file.name,
                    imgElement: img,
                    processedDataUrl: null,
                    eraserMask: document.createElement('canvas'),
                    history: [],
                    historyIndex: -1
                };
                const controls = {
                    brightness: document.getElementById('brightness'),
                    contrast: document.getElementById('contrast'),
                    threshold: document.getElementById('threshold'),
                    shadow: document.getElementById('shadow'),
                    mode: document.querySelectorAll('input[name="mode"]')
                };
                imgObj.settings = {
```

```
                        mode: Array.from(controls.mode).find(r => r.checked).value,
                        brightness: controls.brightness.value,
                        contrast: controls.contrast.value,
                        threshold: controls.threshold.value,
                        shadow: controls.shadow.value
                    };
                    imgObj.eraserMask.width = img.naturalWidth;
                    imgObj.eraserMask.height = img.naturalHeight;

                    images.push(imgObj);
                    saveHistoryState(imgObj); // Save initial state

                    addThumbnail(imgObj, images.length - 1);

                    fileProcessedCount++;
                    if (fileProcessedCount === 1 && currentImageIndex === -1) {
                        // First image loaded
                        workspace.classList.remove('hidden');
                        dropZone.style.display = 'none';
                        selectImage(0);
                    }
                };
                img.src = e.target.result;
            };
            reader.readAsDataURL(file);
        });
}

// --- Rotate ---
const rotateBtn = document.getElementById('rotate-btn');
if (rotateBtn) {
    rotateBtn.addEventListener('click', () => {
        if (currentImageIndex === -1) return;
        const imgObj = images[currentImageIndex];

        const w = imgObj.imgElement.naturalWidth || imgObj.imgElement.width;
        const h = imgObj.imgElement.naturalHeight || imgObj.imgElement.height;

        // Create a new canvas to hold the rotated image
        const rotatedCanvas = document.createElement('canvas');
        rotatedCanvas.width = h;
        rotatedCanvas.height = w;
        const ctx = rotatedCanvas.getContext('2d', { willReadFrequently: true });

        // Move origin to center, rotate, and move back
        ctx.translate(h / 2, w / 2);
```

```
            ctx.rotate(90 * Math.PI / 180);
            ctx.translate(-w / 2, -h / 2);

            // Draw image
            ctx.drawImage(imgObj.imgElement, 0, 0, w, h);

            // Replace imgElement with rotated canvas
            imgObj.imgElement = rotatedCanvas;

            // Reset eraser mask since dimensions swapped
            imgObj.eraserMask = document.createElement('canvas');
            imgObj.eraserMask.width = h;
            imgObj.eraserMask.height = w;

            // If crop mode is on, turn it off to avoid logic issues with rotated dimensions
            if (cropModeEnabled) {
                const cToggle = document.getElementById('crop-toggle');
                if (cToggle) cToggle.click();
            }

            saveHistoryState(imgObj);

            // Update views
            updateOriginalPreview();
            scheduleProcess();
        });
    }

    // --- UI Updates ---

    function addThumbnail(imgObj, index) {
        const div = document.createElement('div');
        div.className = 'thumb-wrapper';
        div.dataset.index = index;

        const img = document.createElement('img');
        img.src = imgObj.imgElement.src;

        const btn = document.createElement('button');
        btn.className = 'remove-thumb';
        btn.innerHTML = '×';
        btn.onclick = (e) => {
            e.stopPropagation();
            removeImage(index);
        };
```

```
    div.appendChild(img);
    div.appendChild(btn);

    div.onclick = () => selectImage(index);
    thumbnailList.appendChild(div);
}

function removeImage(index) {
    images.splice(index, 1);
    updateThumbnails();
    if (images.length === 0) {
        workspace.classList.add('hidden');
        dropZone.style.display = 'block';
        currentImageIndex = -1;
    } else {
        selectImage(Math.min(index, images.length - 1));
    }
}

function updateThumbnails() {
    thumbnailList.innerHTML = '';
    images.forEach((img, i) => addThumbnail(img, i));
    if (currentImageIndex !== -1 && images[currentImageIndex]) {
        thumbnailList.children[currentImageIndex].classList.add('active');
    }
}

function selectImage(index) {
    if (!images[index]) return;

    currentImageIndex = index;
    Array.from(thumbnailList.children).forEach((child, i) => {
        child.classList.toggle('active', i === index);
    });

    if (typeof resetZoom === 'function') resetZoom();
    updateOriginalPreview();
    updateUIFromSettings(images[index].settings);
    updateHistoryButtons();

    // Auto process
    processCurrentImage();
}

function updateOriginalPreview() {
    if (currentImageIndex === -1) return;
```

```
    const imgObj = images[currentImageIndex];
    canvasOriginal.width = imgObj.imgElement.naturalWidth || imgObj.imgElement.width;
    canvasOriginal.height = imgObj.imgElement.naturalHeight || imgObj.imgElement.height;
    ctxOriginal.drawImage(imgObj.imgElement, 0, 0);
    applyZoomTransform();
    if (cropModeEnabled) {
        updateCropOverlay();
    }
}

// --- Zoom Logic ---
const zoomInBtn = document.getElementById('zoom-in-btn');
const zoomOutBtn = document.getElementById('zoom-out-btn');
const zoomFitBtn = document.getElementById('zoom-fit-btn');
const zoomLevelSpan = document.getElementById('zoom-level');
const originalContainer = document.getElementById('original-container');

let zoomScale = 1;
let panX = 0, panY = 0;
let isPanning = false;
let panStartX = 0, panStartY = 0;

function applyZoomTransform() {
    canvasOriginal.style.transform = `scale(${zoomScale}) translate(${panX}px, ${panY}px
    canvasOriginal.style.transformOrigin = 'center center';

    // Redraw overlay to map to new visual coordinates
    if (cropModeEnabled) updateCropOverlay();

    if (zoomLevelSpan) zoomLevelSpan.textContent = Math.round(zoomScale * 100) + '%';
}

function setZoom(newScale) {
    zoomScale = Math.max(0.25, Math.min(5, newScale));
    if (Math.abs(zoomScale - 1) < 0.05) zoomScale = 1; // snap to 100%
    if (zoomScale <= 1) { panX = 0; panY = 0; } // reset pan when zoomed out
    applyZoomTransform();
}

function resetZoom() {
    zoomScale = 1;
    panX = 0;
    panY = 0;
    applyZoomTransform();
    if (cropModeEnabled) updateCropOverlay();
}
```

```
// Mouse wheel zoom
originalContainer.addEventListener('wheel', (e) => {
    e.preventDefault();
    const delta = e.deltaY > 0 ? -0.15 : 0.15;
    setZoom(zoomScale + delta);
}, { passive: false });

// Zoom buttons
if (zoomInBtn) zoomInBtn.addEventListener('click', () => setZoom(zoomScale + 0.25));
if (zoomOutBtn) zoomOutBtn.addEventListener('click', () => setZoom(zoomScale - 0.25));
if (zoomFitBtn) zoomFitBtn.addEventListener('click', () => resetZoom());

// Pan with middle mouse or left mouse when zoomed in
originalContainer.addEventListener('mousedown', (e) => {
    if (zoomScale <= 1) return;
    if (cropModeEnabled && e.target.closest('.crop-handle, .crop-overlay polygon')) retu
    if (eraserModeEnabled) return;
    isPanning = true;
    panStartX = e.clientX - panX;
    panStartY = e.clientY - panY;
    originalContainer.style.cursor = 'grabbing';
    e.preventDefault();
});

document.addEventListener('mousemove', (e) => {
    if (!isPanning) return;
    panX = e.clientX - panStartX;
    panY = e.clientY - panStartY;
    applyZoomTransform();
    if (cropModeEnabled) updateCropOverlay();
});

document.addEventListener('mouseup', () => {
    if (isPanning) {
        isPanning = false;
        originalContainer.style.cursor = zoomScale > 1 ? 'grab' : 'default';
    }
});

// --- Sliders logic ---

function updateSliderValue(sliderId) {
    const slider = document.getElementById(sliderId);
    const valSpan = document.getElementById(`${sliderId}-val`);
    slider.addEventListener('input', () => {
```

```
        valSpan.textContent = slider.value;
    });
}

['brightness', 'contrast', 'threshold', 'shadow'].forEach(updateSliderValue);

// Debounced auto-processing
let processTimeout = null;
function scheduleProcess() {
    if (processTimeout) clearTimeout(processTimeout);
    processTimeout = setTimeout(() => processCurrentImage(), 300);
}

// Auto-process when sliders change
['brightness', 'contrast', 'threshold', 'shadow'].forEach(id => {
    const el = document.getElementById(id);
    el.addEventListener('input', (e) => {
        if (currentImageIndex !== -1) {
            images[currentImageIndex].settings[id] = e.target.value;
            scheduleProcess();
        }
    });
    el.addEventListener('change', () => {
        if (currentImageIndex !== -1) {
            saveHistoryState(images[currentImageIndex]);
        }
    });
});

// Toggle threshold slider visibility based on mode & auto-process
controls.mode.forEach(radio => {
    radio.addEventListener('change', (e) => {
        const val = e.target.value;
        thresholdContainer.style.opacity = val === 'bw' ? '1' : '0.5';
        thresholdContainer.style.pointerEvents = val === 'bw' ? 'auto' : 'none';
        if (currentImageIndex !== -1) {
            images[currentImageIndex].settings.mode = val;
            saveHistoryState(images[currentImageIndex]);
            scheduleProcess();
        }
    });
});

// --- Smart Eraser Logic ---
const eraserToggleBtn = document.getElementById('eraser-toggle');
const eraserClearBtn = document.getElementById('eraser-clear');
```

```
const brushSizeContainer = document.getElementById('brush-size-container');
const brushSizeSlider = document.getElementById('brush-size');
const brushSizeVal = document.getElementById('brush-size-val');

const brushIntensityContainer = document.getElementById('brush-intensity-container');
const brushIntensitySlider = document.getElementById('brush-intensity');
const brushIntensityVal = document.getElementById('brush-intensity-val');

let eraserModeEnabled = false;
let isErasing = false;
let lastEraserPos = null;

// --- Crop Logic ---
const cropToggleBtn = document.getElementById('crop-toggle');
const cropApplyBtn = document.getElementById('crop-apply');
const cropOverlay = document.getElementById('crop-overlay');
const cropPolygon = document.getElementById('crop-polygon');
const cropHandles = [
    document.getElementById('crop-tl'),
    document.getElementById('crop-tr'),
    document.getElementById('crop-br'),
    document.getElementById('crop-bl')
];
let cropModeEnabled = false;
// Points stored in image coordinates (0 to naturalWidth/Height)
let cropPoints = [];
let draggingHandleIndex = -1;

function initCropPoints() {
    if (currentImageIndex === -1) return;
    const imgObj = images[currentImageIndex];
    const w = imgObj.imgElement.naturalWidth || imgObj.imgElement.width;
    const h = imgObj.imgElement.naturalHeight || imgObj.imgElement.height;

    // Initial crop rectangle is 10% from edges
    cropPoints = [
        { x: w * 0.1, y: h * 0.1 },
        { x: w * 0.9, y: h * 0.1 },
        { x: w * 0.9, y: h * 0.9 },
        { x: w * 0.1, y: h * 0.9 }
    ];
    updateCropOverlay();
}

function imageToSvg(imgX, imgY) {
    const container = document.getElementById('original-container');
```

```
    const containerWidth = container.clientWidth;
    const containerHeight = container.clientHeight;

    const cw = canvasOriginal.width;
    const ch = canvasOriginal.height;
    const rectWidth = canvasOriginal.offsetWidth;
    const rectHeight = canvasOriginal.offsetHeight;

    const uX = (imgX / cw) * rectWidth;
    const uY = (imgY / ch) * rectHeight;
    const dx = uX - rectWidth / 2;
    const dy = uY - rectHeight / 2;

    const centerX = containerWidth / 2 + panX * zoomScale;
    const centerY = containerHeight / 2 + panY * zoomScale;

    return {
        x: centerX + dx * zoomScale,
        y: centerY + dy * zoomScale
    };
}

function svgToImage(svgX, svgY) {
    const container = document.getElementById('original-container');
    const containerWidth = container.clientWidth;
    const containerHeight = container.clientHeight;

    const cw = canvasOriginal.width;
    const ch = canvasOriginal.height;
    const rectWidth = canvasOriginal.offsetWidth;
    const rectHeight = canvasOriginal.offsetHeight;

    const centerX = containerWidth / 2 + panX * zoomScale;
    const centerY = containerHeight / 2 + panY * zoomScale;

    const dx = (svgX - centerX) / zoomScale;
    const dy = (svgY - centerY) / zoomScale;

    const uX = dx + rectWidth / 2;
    const uY = dy + rectHeight / 2;

    let imgX = (uX / rectWidth) * cw;
    let imgY = (uY / rectHeight) * ch;

    // Clamp
    imgX = Math.max(0, Math.min(cw, imgX));
```

```
    imgY = Math.max(0, Math.min(ch, imgY));

    return { x: imgX, y: imgY };
}

function updateCropOverlay() {
    if (!cropModeEnabled || cropPoints.length === 0) return;
    const ptsSvg = cropPoints.map(p => imageToSvg(p.x, p.y));

    cropPolygon.setAttribute('points', ptsSvg.map(p => `${p.x},${p.y}`).join(' '));

    for (let i = 0; i < 4; i++) {
        cropHandles[i].setAttribute('cx', ptsSvg[i].x);
        cropHandles[i].setAttribute('cy', ptsSvg[i].y);
    }
}

function toggleCropMode() {
    if (currentImageIndex === -1) return;
    cropModeEnabled = !cropModeEnabled;
    const cropToggleBtn2 = document.getElementById('crop-toggle-2');
    if (cropModeEnabled) {
        if (cropToggleBtn) {
            cropToggleBtn.textContent = '    ';
            cropToggleBtn.classList.replace('secondary-btn', 'primary-btn');
        }
        if (cropToggleBtn2) {
            cropToggleBtn2.textContent = '    ';
            cropToggleBtn2.classList.replace('secondary-btn', 'primary-btn');
        }
        cropApplyBtn.style.display = 'block';
        cropOverlay.style.display = 'block';
        initCropPoints();
    } else {
        if (cropToggleBtn) {
            cropToggleBtn.textContent = '        (   )';
            cropToggleBtn.classList.replace('primary-btn', 'secondary-btn');
        }
        if (cropToggleBtn2) {
            cropToggleBtn2.textContent = '       ';
            cropToggleBtn2.classList.replace('primary-btn', 'secondary-btn');
        }
        cropApplyBtn.style.display = 'none';
        cropOverlay.style.display = 'none';
    }
}
```

```
if (cropToggleBtn) {
    cropToggleBtn.addEventListener('click', toggleCropMode);
}
const cropToggleBtn2 = document.getElementById('crop-toggle-2');
if (cropToggleBtn2) {
    cropToggleBtn2.addEventListener('click', toggleCropMode);
}

    // Crop Handles Dragging
    cropHandles.forEach((handle, index) => {
        handle.addEventListener('mousedown', (e) => {
            draggingHandleIndex = index;
            e.stopPropagation(); // prevent other events
        });
    });

    // Use document for mousemove/mouseup to prevent losing tracking when mouse moves fa
    document.addEventListener('mousemove', (e) => {
        if (draggingHandleIndex === -1 || !cropModeEnabled) return;
        const svgRect = cropOverlay.getBoundingClientRect();
        const svgX = e.clientX - svgRect.left;
        const svgY = e.clientY - svgRect.top;

        cropPoints[draggingHandleIndex] = svgToImage(svgX, svgY);
        updateCropOverlay();
    });

    const stopCropDrag = () => { draggingHandleIndex = -1; };
    document.addEventListener('mouseup', stopCropDrag);

    // Enter key to confirm crop
    document.addEventListener('keydown', (e) => {
        if (e.key === 'Enter' && cropModeEnabled && !cropApplyBtn.disabled) {
            e.preventDefault();
            cropApplyBtn.click();
        }
    });

    cropApplyBtn.addEventListener('click', async () => {
        if (currentImageIndex === -1) return;
        const imgObj = images[currentImageIndex];

        const originalText = cropApplyBtn.textContent;
        cropApplyBtn.textContent = '     ...  ';
        cropApplyBtn.disabled = true;
```

```
// Allow UI to render the loading text
await new Promise(r => setTimeout(r, 50));

try {
    // Calculate destination size based on max width/height of the polygon
    const w1 = Math.hypot(cropPoints[0].x - cropPoints[1].x, cropPoints[0].y - c
    const w2 = Math.hypot(cropPoints[2].x - cropPoints[3].x, cropPoints[2].y - c
    const dstW = Math.max(10, Math.round(Math.max(w1, w2)));

    const h1 = Math.hypot(cropPoints[1].x - cropPoints[2].x, cropPoints[1].y - c
    const h2 = Math.hypot(cropPoints[3].x - cropPoints[0].x, cropPoints[3].y - c
    const dstH = Math.max(10, Math.round(Math.max(h1, h2)));

    if (dstW < 10 || dstH < 10) {
        alert('            !  ';(
        return;
    }

    // Apply warp
    const warpedCanvas = await window.DocProcessor.perspectiveWarp(imgObj.imgEle

    // Replace imgElement with warped canvas directly (Canvas API works as image
    imgObj.imgElement = warpedCanvas;

    // Reset eraser mask for this image since size changed
    imgObj.eraserMask = document.createElement('canvas');
    imgObj.eraserMask.width = dstW;
    imgObj.eraserMask.height = dstH;

    // Turn off crop mode
    if (cropModeEnabled) {
        cropToggleBtn.click();
    }

    saveHistoryState(imgObj);

    // Update views
    updateOriginalPreview();
    scheduleProcess();
} catch (err) {
    console.error('Crop Error:', err);
    alert('       !   ';(
} finally {
    cropApplyBtn.textContent = originalText;
    cropApplyBtn.disabled = false;
```

```
        }
    });

if (eraserToggleBtn) {
    eraserToggleBtn.addEventListener('click', () => {
        eraserModeEnabled = !eraserModeEnabled;
        if (eraserModeEnabled) {
            eraserToggleBtn.textContent = '        (  )';
            eraserToggleBtn.classList.replace('secondary-btn', 'primary-btn');
            eraserClearBtn.style.display = 'block';
            brushSizeContainer.style.display = 'block';
            brushIntensityContainer.style.display = 'block';
            resultContainer.classList.add('eraser-active');
        } else {
            eraserToggleBtn.textContent = '        (  )';
            eraserToggleBtn.classList.replace('primary-btn', 'secondary-btn');
            eraserClearBtn.style.display = 'none';
            brushSizeContainer.style.display = 'none';
            brushIntensityContainer.style.display = 'none';
            resultContainer.classList.remove('eraser-active');
        }
    });

    brushSizeSlider.addEventListener('input', () => {
        brushSizeVal.textContent = brushSizeSlider.value;
    });

    brushIntensitySlider.addEventListener('input', () => {
        brushIntensityVal.textContent = brushIntensitySlider.value;
    });

    eraserClearBtn.addEventListener('click', () => {
        if (currentImageIndex === -1) return;
        const imgObj = images[currentImageIndex];
        const maskCtx = imgObj.eraserMask.getContext('2d');
        maskCtx.clearRect(0, 0, imgObj.eraserMask.width, imgObj.eraserMask.height);
        saveHistoryState(imgObj);
        scheduleProcess();
    });

    function getMousePos(canvas, evt) {
        const container = document.getElementById('original-container');
        const containerRect = container.getBoundingClientRect();
        const containerWidth = container.clientWidth;
        const containerHeight = container.clientHeight;
```

```
    const cw = canvas.width;
    const ch = canvas.height;
    const rectWidth = canvas.offsetWidth;
    const rectHeight = canvas.offsetHeight;

    const centerX = containerWidth / 2 + panX * zoomScale;
    const centerY = containerHeight / 2 + panY * zoomScale;

    const mouseX = evt.clientX - containerRect.left;
    const mouseY = evt.clientY - containerRect.top;

    const dx = (mouseX - centerX) / zoomScale;
    const dy = (mouseY - centerY) / zoomScale;

    const uX = dx + rectWidth / 2;
    const uY = dy + rectHeight / 2;

    const x = (uX / rectWidth) * cw;
    const y = (uY / rectHeight) * ch;

    return { x, y };
}

canvasResult.addEventListener('mousedown', (e) => {
    if (!eraserModeEnabled || currentImageIndex === -1) return;
    isErasing = true;
    lastEraserPos = getMousePos(canvasResult, e);
});

canvasResult.addEventListener('mousemove', (e) => {
    if (!isErasing || !eraserModeEnabled || currentImageIndex === -1) return;
    const pos = getMousePos(canvasResult, e);
    const brushSize = parseInt(brushSizeSlider.value);
    const brushIntensity = parseInt(brushIntensitySlider.value) / 100;
    const imgObj = images[currentImageIndex];
    const maskCtx = imgObj.eraserMask.getContext('2d');

    maskCtx.strokeStyle = `rgba(255, 255, 255, ${brushIntensity})`;
    maskCtx.lineWidth = brushSize;
    maskCtx.lineCap = 'round';
    maskCtx.lineJoin = 'round';
    maskCtx.beginPath();
    maskCtx.moveTo(lastEraserPos.x, lastEraserPos.y);
    maskCtx.lineTo(pos.x, pos.y);
    maskCtx.stroke();
```

```
            ctxResult.strokeStyle = `rgba(255, 255, 255, ${Math.max(0.1, brushIntensity / 2)
            ctxResult.lineWidth = brushSize;
            ctxResult.lineCap = 'round';
            ctxResult.lineJoin = 'round';
            ctxResult.beginPath();
            ctxResult.moveTo(lastEraserPos.x, lastEraserPos.y);
            ctxResult.lineTo(pos.x, pos.y);
            ctxResult.stroke();

            lastEraserPos = pos;
        });

        const stopErasing = () => {
            if (isErasing) {
                isErasing = false;
                if (currentImageIndex !== -1) {
                    saveHistoryState(images[currentImageIndex]);
                }
                scheduleProcess();
            }
        };

        canvasResult.addEventListener('mouseup', stopErasing);
        canvasResult.addEventListener('mouseleave', stopErasing);
    }

    // --- Background Removal ---
    const bgRemoveBtn = document.getElementById('bg-remove-btn');
    const bgToleranceSlider = document.getElementById('bg-tolerance');
    const bgToleranceVal = document.getElementById('bg-tolerance-val');
    const bgSmoothSlider = document.getElementById('bg-smooth');
    const bgSmoothVal = document.getElementById('bg-smooth-val');

    if (bgToleranceSlider) {
        bgToleranceSlider.addEventListener('input', () => {
            bgToleranceVal.textContent = bgToleranceSlider.value;
        });
    }
    if (bgSmoothSlider) {
        bgSmoothSlider.addEventListener('input', () => {
            bgSmoothVal.textContent = bgSmoothSlider.value;
        });
    }

    if (bgRemoveBtn) {
        bgRemoveBtn.addEventListener('click', async () => {
```

```
        if (currentImageIndex === -1) return;
        const imgObj = images[currentImageIndex];

        bgRemoveBtn.textContent = '\u23f3 \u062f\u0631 \u062d\u0627\u0644 \u062d\u0630\u
        bgRemoveBtn.disabled = true;

        try {
            const tolerance = parseInt(bgToleranceSlider.value);
            const edgeSmooth = parseInt(bgSmoothSlider.value);

            const resultCanvas = await window.DocProcessor.removeBackground(
                imgObj.imgElement, tolerance, edgeSmooth
            );

            imgObj.imgElement = resultCanvas;

            // Reset eraser mask for new dimensions
            imgObj.eraserMask = document.createElement('canvas');
            imgObj.eraserMask.width = resultCanvas.width;
            imgObj.eraserMask.height = resultCanvas.height;

            saveHistoryState(imgObj);
            updateOriginalPreview();
            scheduleProcess();

            // Switch to PNG format since we have transparency
            if (exportFormat) exportFormat.value = 'image/png';
        } catch (err) {
            console.error('Background Removal Error:', err);
            alert('\u062e\u0637\u0627 \u062f\u0631 \u062d\u0630\u0641 \u0628\u06a9\u06a
        } finally {
            bgRemoveBtn.textContent = '\u062d\u0630\u0641 \u0628\u06a9\u06af\u0631\u0627
            bgRemoveBtn.disabled = false;
        }
    });
}

// --- Transfer to Composer ---
const openComposerLink = document.getElementById('open-composer-link');
if (openComposerLink) {
    openComposerLink.addEventListener('click', () => {
        if (currentImageIndex !== -1 && images[currentImageIndex]) {
            const imgObj = images[currentImageIndex];
            const dataUrl = imgObj.processedDataUrl || canvasResult.toDataURL('image/png
            try {
                localStorage.setItem('composer_import_image', JSON.stringify({
```

```
                    name: imgObj.name || '        ',
                    dataUrl: dataUrl
                }));
            } catch (e) {
                console.warn('Could not store image for composer:', e);
            }
        }
    });
}

// --- Processing ---

async function processCurrentImage() {
    if (currentImageIndex === -1 || isProcessing) return;

    isProcessing = true;
    resultContainer.classList.add('loading');

    const imgObj = images[currentImageIndex];
    const options = {
        ...imgObj.settings,
        eraserMask: imgObj.eraserMask
    };

    try {
        const processedCanvas = await window.DocProcessor.process(imgObj.imgElement, opt

        canvasResult.width = processedCanvas.width;
        canvasResult.height = processedCanvas.height;
        ctxResult.drawImage(processedCanvas, 0, 0);

        imgObj.processedDataUrl = canvasResult.toDataURL(exportFormat.value, 0.9);

    } catch (error) {
        console.error("Processing failed", error);
    } finally {
        isProcessing = false;
        resultContainer.classList.remove('loading');
    }
}

processBtn.addEventListener('click', processCurrentImage);

// --- Downloads ---

downloadSingleBtn.addEventListener('click', () => {
```

```
    if (currentImageIndex === -1 || !images[currentImageIndex].processedDataUrl) return;

    const imgObj = images[currentImageIndex];
    const format = exportFormat.value === 'image/jpeg' ? 'jpg' : 'png';
    const link = document.createElement('a');
    link.download = `scanned_${imgObj.name.split('.')[0]}.${format}`;
    link.href = imgObj.processedDataUrl;
    link.click();
});

downloadAllBtn.addEventListener('click', async () => {
    if (images.length === 0) return;

    downloadAllBtn.textContent = '       ...  ';
    downloadAllBtn.disabled = true;

    const zip = new JSZip();
    const format = exportFormat.value === 'image/jpeg' ? 'jpg' : 'png';

    // Ensure all are processed

    for (let i = 0; i < images.length; i++) {
        let imgObj = images[i];

        // Re-apply specific options for the batch export with each image's mask
        const exportOptions = { ...imgObj.settings, eraserMask: imgObj.eraserMask };
        const processedCanvas = await window.DocProcessor.process(imgObj.imgElement, exp
        const dataUrl = processedCanvas.toDataURL(exportFormat.value, 0.9);
        const base64Data = dataUrl.replace(/^data:image\/(png|jpeg)base64,/, "");
        zip.file(`scanned_${imgObj.name.split('.')[0]}.${format}`, base64Data, {base64:
    }

    zip.generateAsync({type:"blob"}).then(function(content) {
        const link = document.createElement('a');
        link.href = URL.createObjectURL(content);
        link.download = "scanned_documents.zip";
        link.click();

        downloadAllBtn.textContent = '       (ZIP)';
        downloadAllBtn.disabled = false;
    });
});

// --- PDF Download ---
const downloadPdfBtn = document.getElementById('download-pdf-btn');
```

```
// Page sizes in mm
const PDF_SIZES = {
    'id-card': { w: 85.6, h: 53.98, label: '     ' {,
    'a5':      { w: 148,  h: 210,   label: 'A5' },
    'a4':      { w: 210,  h: 297,   label: 'A4' },
    'a3':      { w: 297,  h: 420,   label: 'A3' }
};

downloadPdfBtn.addEventListener('click', async () => {
    const pdfSizeRadio = document.querySelector('input[name="pdf-size"]:checked');
    if (!pdfSizeRadio) {
        alert('        PDF     !  ';(
        return;
    }
    if (images.length === 0) {
        alert('          !   ';(
        return;
    }

    const sizeKey = pdfSizeRadio.value;
    const size = PDF_SIZES[sizeKey];

    downloadPdfBtn.textContent = '         PDF...';
    downloadPdfBtn.disabled = true;

    try {
        const jsPDF = window.jspdf ? window.jspdf.jsPDF : window.jsPDF;
        if (!jsPDF) {
            alert('          PDF          .                              .  ';(
            return;
        }

        // For ID Card, we output an A4 page to prevent printer scaling issues.
        const isIdCard = (sizeKey === 'id-card');

        let pageW = isIdCard ? PDF_SIZES['a4'].w : size.w;
        let pageH = isIdCard ? PDF_SIZES['a4'].h : size.h;

        const orientation = pageW > pageH ? 'landscape' : 'portrait';

        const doc = new jsPDF({
            orientation: orientation,
            unit: 'mm',
            format: [pageW, pageH]
        });
```

```
let currentIdCardY = 20; // 2cm margin from top

for (let i = 0; i < images.length; i++) {
    if (i > 0) {
        if (isIdCard) {
            // For ID cards, only add a new page if we run out of vertical space
            if (currentIdCardY + size.h + 10 > pageH - 20) {
                doc.addPage([pageW, pageH], orientation);
                currentIdCardY = 20;
            }
        } else {
            // For A4/A5, always add a new page
            doc.addPage([pageW, pageH], orientation);
        }
    }

    const imgObj = images[i];
    const exportOptions = { ...imgObj.settings, eraserMask: imgObj.eraserMask };
    const processedCanvas = await window.DocProcessor.process(imgObj.imgElement,

    const imgW = processedCanvas.width;
    const imgH = processedCanvas.height;
    const imgRatio = imgW / imgH;

    let drawW, drawH, offsetX, offsetY;

    if (isIdCard) {
        // Force the image into the exact physical dimensions of an ID card (85.
        // We maintain aspect ratio so it doesn't distort if poorly cropped
        const targetRatio = size.w / size.h;
        if (imgRatio > targetRatio) {
            drawW = size.w;
            drawH = size.w / imgRatio;
        } else {
            drawH = size.h;
            drawW = size.h * imgRatio;
        }
        // Center relative to A5 width (148mm) so it prints safely on BOTH A4 an
        // A5 width is 148. So center is at (148 - drawW) / 2 = ~31mm.
        offsetX = Math.max(10, (148 - drawW) / 2);
        offsetY = currentIdCardY;

        // Increment Y for the next image on this page
        currentIdCardY += drawH + 10; // 10mm gap between cards
    } else {
        const margin = 5;
```

```
                    const safeW = pageW - (margin * 2);
                    const safeH = pageH - (margin * 2);
                    const safeRatio = safeW / safeH;

                    if (imgRatio > safeRatio) {
                        drawW = safeW;
                        drawH = safeW / imgRatio;
                        offsetX = margin;
                        offsetY = margin + (safeH - drawH) / 2;
                    } else {
                        drawH = safeH;
                        drawW = safeH * imgRatio;
                        offsetX = margin + (safeW - drawW) / 2;
                        offsetY = margin;
                    }
                }

                // Pass canvas directly to avoid huge base64 strings and memory issues
                doc.addImage(processedCanvas, 'JPEG', offsetX, offsetY, drawW, drawH);
            }

            doc.save(`scanned_${size.label}.pdf`);

        } catch (err) {
            console.error('PDF Error:', err);
            alert('        PDF: ' + (err.message || err));
        } finally {
            downloadPdfBtn.textContent = '    PDF';
            downloadPdfBtn.disabled = false;
        }
    });
});
```

:لای‌ف background.js

```
chrome.action.onClicked.addListener((tab) => {
  chrome.tabs.create({
    url: 'index.html'
  });
});
```

:لای‌ف composer.css

```
/* Composer Design System - Matches styles.css with Windows Desktop aesthetic */
```

```
@font-face {
    font-family: 'Vazirmatn';
    src: url('fonts/Vazirmatn-Regular.woff2') format('woff2');
    font-weight: 400;
    font-style: normal;
    font-display: swap;
}

@font-face {
    font-family: 'Vazirmatn';
    src: url('fonts/Vazirmatn-Bold.woff2') format('woff2');
    font-weight: 700;
    font-style: normal;
    font-display: swap;
}

:root {
    --bg-color: #f0f0f0;
    --panel-bg: #ffffff;
    --panel-border: #cccccc;
    --text-primary: #000000;
    --text-secondary: #333333;
    --accent: #0078d7;
    --accent-hover: #005a9e;
    --danger: #d13438;
    --danger-hover: #a82327;

    --btn-bg: #e1e1e1;
    --btn-border: #adadad;
    --btn-hover-bg: #e5f1fb;
    --btn-hover-border: #0078d7;

    --input-border: #cccccc;
    --input-focus: #0078d7;

    --ruler-size: 25px;
    --ruler-bg: #f5f5f5;
    --ruler-border: #d0d0d0;
}

* {
    box-sizing: border-box;
    margin: 0;
    padding: 0;
}
```

```
body {
    font-family: Tahoma, 'Segoe UI', 'Vazirmatn', sans-serif;
    background-color: var(--bg-color);
    color: var(--text-primary);
    min-height: 100vh;
    font-size: 14px;
    direction: rtl;
    display: flex;
    flex-direction: column;
}

/* Header */
.app-header {
    background: var(--panel-bg);
    border-bottom: 1px solid var(--panel-border);
    padding: 10px 20px;
    display: flex;
    justify-content: space-between;
    align-items: center;
    box-shadow: 0 1px 3px rgba(0, 0, 0, 0.05);
    z-index: 20;
}

.header-title-group h1 {
    font-size: 1.3rem;
    font-weight: 600;
    color: var(--text-primary);
}

.header-title-group p {
    font-size: 0.85rem;
    color: var(--text-secondary);
    margin-top: 2px;
}

.nav-back-link {
    display: inline-flex;
    align-items: center;
    gap: 6px;
    padding: 6px 14px;
    background: var(--btn-bg);
    border: 1px solid var(--btn-border);
    color: var(--text-primary);
    text-decoration: none;
    font-size: 0.85rem;
    border-radius: 2px;
```

```
    transition: background 0.15s, border-color 0.15s;
}

.nav-back-link:hover {
    background: var(--btn-hover-bg);
    border-color: var(--btn-hover-border);
    color: var(--accent-hover);
}

/* Main Composer Layout */
.composer-container {
    display: flex;
    flex-direction: row; /* In RTL: first child is on the Right! */
    flex: 1;
    height: calc(100vh - 65px);
    overflow: hidden;
    position: relative;
}

/* Sidebar (Right side in RTL) */
.composer-sidebar {
    width: 320px;
    min-width: 320px;
    background: var(--panel-bg);
    border-left: 1px solid var(--panel-border);
    display: flex;
    flex-direction: column;
    gap: 12px;
    padding: 12px;
    overflow-y: auto;
    z-index: 10;
}

/* Control Groups */
.control-group {
    border: 1px solid var(--panel-border);
    padding: 12px 10px 10px 10px;
    background: #fbfbfb;
    border-radius: 2px;
    position: relative;
    margin-top: 8px;
}

.control-group h3 {
    font-size: 0.85rem;
    font-weight: bold;
```

```
    position: absolute;
    top: -10px;
    right: 10px;
    background: #fbfbfb;
    padding: 0 6px;
    color: var(--text-secondary);
}

/* Radio and Checkbox Options */
.page-controls {
    display: flex;
    flex-direction: column;
    gap: 8px;
}

.radio-group-inline {
    display: flex;
    gap: 12px;
    flex-wrap: wrap;
}

.radio-label, .checkbox-label {
    display: inline-flex;
    align-items: center;
    gap: 6px;
    cursor: pointer;
    font-size: 0.85rem;
    user-select: none;
}

.radio-label input[type="radio"],
.checkbox-label input[type="checkbox"] {
    margin: 0;
    cursor: pointer;
}

/* Buttons */
.btn {
    width: 100%;
    padding: 6px 12px;
    font-family: inherit;
    font-size: 0.85rem;
    border: 1px solid var(--btn-border);
    background: var(--btn-bg);
    color: var(--text-primary);
    cursor: pointer;
```

```
    text-align: center;
    display: inline-flex;
    align-items: center;
    justify-content: center;
    gap: 6px;
    border-radius: 2px;
    transition: all 0.15s ease;
}

.btn:hover:not(:disabled) {
    background: var(--btn-hover-bg);
    border-color: var(--btn-hover-border);
}

.btn:disabled {
    opacity: 0.5;
    cursor: not-allowed;
}

.primary-btn {
    background: var(--accent);
    color: #ffffff;
    border-color: var(--accent);
}

.primary-btn:hover:not(:disabled) {
    background: var(--accent-hover);
    border-color: var(--accent-hover);
}

.danger-btn {
    background: #fff;
    color: var(--danger);
    border-color: var(--danger);
}

.danger-btn:hover:not(:disabled) {
    background: var(--danger);
    color: #ffffff;
}

.btn svg {
    flex-shrink: 0;
}

/* Image List */
```

```
.image-list-container {
    display: flex;
    flex-direction: column;
    gap: 8px;
    max-height: 240px;
    overflow-y: auto;
    border: 1px solid var(--input-border);
    background: #ffffff;
    padding: 6px;
    border-radius: 2px;
}

.image-list-empty {
    text-align: center;
    color: #888888;
    font-size: 0.8rem;
    padding: 20px 8px;
}

.image-item {
    display: flex;
    align-items: center;
    gap: 8px;
    padding: 6px;
    background: #f9f9f9;
    border: 1px solid var(--panel-border);
    border-radius: 2px;
    cursor: pointer;
    transition: background 0.15s, border-color 0.15s;
}

.image-item:hover {
    background: #f0f7ff;
    border-color: var(--accent);
}

.image-item.active {
    background: #e5f1fb;
    border-color: var(--accent);
    box-shadow: inset 2px 0 0 var(--accent);
}

.image-item-thumb {
    width: 44px;
    height: 44px;
    object-fit: cover;
```

```
    background: #e9e9e9;
    border: 1px solid #ccc;
    flex-shrink: 0;
}

.image-item-info {
    flex: 1;
    min-width: 0;
    display: flex;
    flex-direction: column;
    gap: 2px;
}

.image-item-title {
    font-size: 0.8rem;
    font-weight: 600;
    white-space: nowrap;
    overflow: hidden;
    text-overflow: ellipsis;
}

.image-item-dims {
    font-size: 0.75rem;
    color: #666;
    direction: ltr;
    text-align: right;
}

.image-item-actions {
    display: flex;
    align-items: center;
    gap: 4px;
}

.image-item-delete {
    background: transparent;
    border: none;
    color: var(--danger);
    cursor: pointer;
    padding: 4px;
    display: flex;
    align-items: center;
    justify-content: center;
    border-radius: 2px;
}
```

```
.image-item-delete:hover {
    background: #fee;
}

/* Viewport / Work Area */
.composer-viewport {
    flex: 1;
    display: flex;
    flex-direction: column;
    background: #505050; /* Neutral dark grey background for artboard */
    position: relative;
    overflow: hidden;
    min-width: 0;
}

.viewport-toolbar {
    background: #e6e6e6;
    border-bottom: 1px solid var(--panel-border);
    padding: 4px 12px;
    display: flex;
    justify-content: space-between;
    align-items: center;
    font-size: 0.8rem;
    z-index: 10;
}

.viewport-status {
    color: #333;
    font-size: 0.8rem;
    display: flex;
    gap: 15px;
}

.zoom-controls {
    display: flex;
    align-items: center;
    gap: 4px;
}

.zoom-btn {
    width: 24px;
    height: 24px;
    padding: 0;
    font-size: 14px;
    font-weight: bold;
    display: inline-flex;
```

```
    align-items: center;
    justify-content: center;
    background: var(--btn-bg);
    border: 1px solid var(--btn-border);
    cursor: pointer;
    border-radius: 2px;
}

.zoom-btn:hover {
    background: var(--btn-hover-bg);
    border-color: var(--btn-hover-border);
}

.zoom-level {
    font-size: 12px;
    min-width: 48px;
    text-align: center;
    font-weight: 500;
}

/* Rulers and Canvas Wrapper */
.composer-canvas-area {
    position: relative;
    flex: 1;
    overflow: hidden;
    display: grid;
    grid-template-columns: var(--ruler-size) 1fr;
    grid-template-rows: var(--ruler-size) 1fr;
    background: #555555;
}

.ruler-corner {
    grid-column: 1 / 2;
    grid-row: 1 / 2;
    background: #e0e0e0;
    border-right: 1px solid var(--ruler-border);
    border-bottom: 1px solid var(--ruler-border);
    z-index: 5;
    display: flex;
    align-items: center;
    justify-content: center;
    font-size: 9px;
    color: #666;
    user-select: none;
}
```

```
.ruler-h-wrapper {
    grid-column: 2 / 3;
    grid-row: 1 / 2;
    position: relative;
    overflow: hidden;
    background: var(--ruler-bg);
    border-bottom: 1px solid var(--ruler-border);
    z-index: 4;
}

.ruler-v-wrapper {
    grid-column: 1 / 2;
    grid-row: 2 / 3;
    position: relative;
    overflow: hidden;
    background: var(--ruler-bg);
    border-right: 1px solid var(--ruler-border);
    z-index: 4;
}

.ruler-h, .ruler-v {
    position: absolute;
    top: 0;
    left: 0;
    pointer-events: none;
}

/* Paper Canvas Area */
.paper-canvas-wrapper {
    grid-column: 2 / 3;
    grid-row: 2 / 3;
    position: relative;
    overflow: hidden;
    cursor: default;
    background: #606060;
    display: flex;
    align-items: center;
    justify-content: center;
}

#paper-canvas {
    display: block;
    box-shadow: 0 4px 20px rgba(0, 0, 0, 0.45), 0 0 1px rgba(0,0,0,0.6);
    background: #ffffff;
    user-select: none;
}
```

```
/* Floating Badges and Helpers */
.size-badge {
    position: absolute;
    background: rgba(0, 120, 215, 0.95);
    color: #ffffff;
    padding: 3px 8px;
    font-size: 11px;
    font-family: Tahoma, 'Segoe UI', sans-serif;
    border-radius: 3px;
    pointer-events: none;
    z-index: 50;
    box-shadow: 0 2px 6px rgba(0,0,0,0.3);
    white-space: nowrap;
    direction: ltr;
    transform: translate(-50%, -100%);
    margin-top: -8px;
}

/* Guidelines */
.guideline {
    position: absolute;
    pointer-events: none;
    z-index: 30;
}

.guideline.horizontal {
    left: 0;
    right: 0;
    height: 0;
    border-top: 1px dashed #0078d7;
}

.guideline.vertical {
    top: 0;
    bottom: 0;
    width: 0;
    border-left: 1px dashed #0078d7;
}

/* Responsive */
@media (max-width: 900px) {
    .composer-container {
        flex-direction: column;
        height: auto;
    }
```

```
    .composer-sidebar {
        width: 100%;
        min-width: 100%;
        border-left: none;
        border-bottom: 1px solid var(--panel-border);
        height: auto;
        max-height: 380px;
    }
    .composer-viewport {
        height: 600px;
    }
}

/* Selected Image Dimension Tools */
.dimension-inputs-grid {
    display: grid;
    grid-template-columns: 1fr 1fr;
    gap: 8px;
    margin-top: 6px;
}

.dim-field {
    display: flex;
    flex-direction: column;
    gap: 3px;
}

.dim-field label {
    font-size: 11px;
    color: var(--text-secondary);
    font-weight: 500;
}

.dim-field input {
    width: 100%;
    padding: 5px 8px;
    border: 1px solid var(--input-border);
    border-radius: 2px;
    font-size: 13px;
    direction: ltr;
    text-align: center;
    background: #fff;
}

.dim-field input:focus {
    border-color: var(--input-focus);
```

```
    outline: none;
}

.action-buttons-row {
    display: flex;
    gap: 6px;
    flex-wrap: wrap;
}

.btn-sm {
    padding: 4px 8px !important;
    font-size: 11px !important;
    flex: 1;
    min-width: 0;
    justify-content: center;
    white-space: nowrap;
}

.slider-container {
    margin-bottom: 6px;
}

.slider-header {
    display: flex;
    justify-content: space-between;
    align-items: center;
    margin-bottom: 2px;
    font-size: 12px;
}

.slider-val {
    font-weight: bold;
    color: var(--accent);
}

/* Drag over dropzone styling */
.paper-canvas-wrapper.drag-over {
    box-shadow: inset 0 0 0 3px var(--accent), 0 4px 20px rgba(0, 120, 215, 0.3) !important;
}
```

composer.html :فایل

```
<!DOCTYPE html>
<html lang="fa" dir="rtl">
<head>
```

```
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>       -           <title/>
    <link rel="stylesheet" href="composer.css">
    <script src="jspdf.umd.min.js"></script>
    <script src="processor.js"></script>
</head>
<body>
    <header class="app-header">
        <div class="header-title-group">
            <h1>            <h1/>
            <p>                                          PDF</p>
        </div>
        <div>
            <a href="index.html" class="nav-back-link">
                <svg viewBox="0 0 24 24" width="16" height="16" stroke="currentColor" stroke
                    <line x1="19" y1="12" x2="5" y2="12"></line>
                    <polyline points="12 19 5 12 12 5"></polyline>
                </svg>

            </a>
        </div>
    </header>

    <main class="composer-container">
        <!-- Sidebar with controls (Right side in RTL) -->
        <aside class="composer-sidebar">
            <!-- Paper Size -->
            <div class="control-group">
                <h3>    <h3/>
                <div class="page-controls">
                    <div class="radio-group-inline">
                        <label class="radio-label">
                            <input type="radio" name="paper-size" value="a3">
                            <span>A3</span>
                        </label>
                        <label class="radio-label">
                            <input type="radio" name="paper-size" value="a4" checked>
                            <span>A4</span>
                        </label>
                        <label class="radio-label">
                            <input type="radio" name="paper-size" value="a5">
                            <span>A5</span>
                        </label>
                    </div>
                </div>
```

```
</div>

<!-- Orientation -->
<div class="control-group">
    <h3>   <h3/>
    <div class="page-controls">
        <div class="radio-group-inline">
            <label class="radio-label">
                <input type="radio" name="orientation" value="portrait" checked>
                <span>   </span>
            </label>
            <label class="radio-label">
                <input type="radio" name="orientation" value="landscape">
                <span>  </span>
            </label>
        </div>
    </div>
</div>

<!-- Display / Snap Options -->
<div class="control-group">
    <h3>      <h3/>
    <div class="page-controls">
        <label class="checkbox-label">
            <input type="checkbox" id="toggle-grid">
            <span>            <span/>
        </label>
        <label class="checkbox-label">
            <input type="checkbox" id="toggle-guidelines" checked>
            <span>          <span/>(    )
        </label>
    </div>
</div>

<!-- Image Actions -->
<div class="control-group">
    <h3>   </h3>
    <input type="file" id="image-file-input" multiple accept="image/*" style="di
    <button type="button" class="btn primary-btn" id="add-image-btn" style="marg
        <svg viewBox="0 0 24 24" width="16" height="16" stroke="currentColor" st
            <line x1="12" y1="5" x2="12" y2="19"></line>
            <line x1="5" y1="12" x2="19" y2="12"></line>
        </svg>

    </button>
```

```
    <div class="image-list-container" id="image-list">
        <div class="image-list-empty" id="image-list-empty">

        </div>
</div>

<!-- Selected Image Tools & Measurement -->
<div class="control-group" id="selected-image-group" style="display: none;">
    <h3>         <h3/>
    <div class="dimension-inputs-grid">
        <div class="dim-field">
            <label for="img-width-input">   (cm)</label>
            <input type="number" id="img-width-input" step="0.1" min="0.5" max='
        </div>
        <div class="dim-field">
            <label for="img-height-input">    (cm)</label>
            <input type="number" id="img-height-input" step="0.1" min="0.5" max=
        </div>
    </div>

    <!-- Position X and Y in cm (allows negative to overhang off paper) -->
    <div class="dimension-inputs-grid" style="margin-top: 6px;">
        <div class="dim-field">
            <label for="img-posx-input">       X (cm)</label>
            <input type="number" id="img-posx-input" step="0.1" title="
        </div>
        <div class="dim-field">
            <label for="img-posy-input">        Y (cm)</label>
            <input type="number" id="img-posy-input" step="0.1" title="
        </div>
    </div>

    <label class="checkbox-label" style="margin-top: 6px;">
        <input type="checkbox" id="lock-aspect-ratio" checked>
        <span>      <span/>
    </label>

    <!-- Equalize size buttons -->
    <div class="action-buttons-row" style="margin-top: 8px;">
        <button type="button" class="btn secondary-btn btn-sm" id="match-height-

        </button>
        <button type="button" class="btn secondary-btn btn-sm" id="match-width-b

        </button>
    </div>
```

```
<!-- Background Removal inside Composer -->
<div style="margin-top: 10px; border-top: 1px dashed var(--panel-border); pa
    <button type="button" class="btn secondary-btn btn-sm" id="composer-remo

    </button>
    <div class="slider-container" id="composer-tolerance-container" style="d
        <div class="slider-header" style="font-size: 11px;">
            <label for="composer-bg-tol">    <label/>
            <span id="composer-bg-tol-val" class="slider-val">30</span>
        </div>
        <input type="range" id="composer-bg-tol" min="5" max="80" value="30"
    </div>
</div>

<!-- Photoshop-style Crop Tool   ) (    -->
<div style="margin-top: 10px; border-top: 1px dashed var(--panel-border); pa
    <button type="button" class="btn secondary-btn btn-sm" id="toggle-crop-b
                  (Crop)
    </button>
    <div id="crop-panel" style="display: none; margin-top: 8px; background:
        <p style="font-size: 11px; color: #444; margin-bottom: 6px; font-wei
        <div class="action-buttons-row" style="margin-bottom: 8px;">
            <button type="button" class="btn secondary-btn btn-sm" id="crop-

            </button>
            <button type="button" class="btn secondary-btn btn-sm" id="crop-

            </button>
            <button type="button" class="btn secondary-btn btn-sm" id="crop-

            </button>
            <button type="button" class="btn secondary-btn btn-sm" id="crop-

            </button>
        </div>

        <div class="slider-container">
            <div class="slider-header" style="font-size: 11px;">
                <label for="crop-left-slider">     <label/>
                <span id="crop-left-val" class="slider-val">0%</span>
            </div>
            <input type="range" id="crop-left-slider" min="0" max="85" value
        </div>

        <div class="slider-container">
```

```
            <div class="slider-header" style="font-size: 11px;">
                <label for="crop-right-slider">    <label/>
                <span id="crop-right-val" class="slider-val">0%</span>
            </div>
            <input type="range" id="crop-right-slider" min="0" max="85" valu
        </div>

        <div class="slider-container">
            <div class="slider-header" style="font-size: 11px;">
                <label for="crop-top-slider">    <label/>
                <span id="crop-top-val" class="slider-val">0%</span>
            </div>
            <input type="range" id="crop-top-slider" min="0" max="85" value=
        </div>

        <div class="slider-container">
            <div class="slider-header" style="font-size: 11px;">
                <label for="crop-bottom-slider">    <label/>
                <span id="crop-bottom-val" class="slider-val">0%</span>
            </div>
            <input type="range" id="crop-bottom-slider" min="0" max="85" val
        </div>

        <div class="action-buttons-row" style="margin-top: 8px;">
            <button type="button" class="btn primary-btn btn-sm" id="apply-c

            </button>
            <button type="button" class="btn secondary-btn btn-sm" id="reset

            </button>
        </div>
    </div>
</div>

<!-- Layer and Rotation buttons -->
<div class="action-buttons-row" style="margin-top: 8px;">
    <button type="button" class="btn secondary-btn btn-sm" id="rotate-image-
              °
    </button>
    <button type="button" class="btn secondary-btn btn-sm" id="bring-to-fron

    </button>
    <button type="button" class="btn secondary-btn btn-sm" id="send-to-back-

    </button>
</div>
```

```
    </div>

    <!-- Layout and Export Actions -->
    <div class="control-group">
        <h3>      <h3/>
        <div style="display: flex; flex-direction: column; gap: 8px;">
            <button type="button" class="btn secondary-btn" id="auto-layout-btn" tit
                <svg viewBox="0 0 24 24" width="16" height="16" stroke="currentColor
                    <rect x="3" y="3" width="7" height="7"></rect>
                    <rect x="14" y="3" width="7" height="7"></rect>
                    <rect x="14" y="14" width="7" height="7"></rect>
                    <rect x="3" y="14" width="7" height="7"></rect>
                </svg>

            </button>
            <button type="button" class="btn primary-btn" id="export-pdf-btn">
                <svg viewBox="0 0 24 24" width="16" height="16" stroke="currentColor
                    <path d="M21 15v4a2 2 0 0 1-2 2H5a2 2 0 0 1-2-2v-4"></path>
                    <polyline points="7 10 12 15 17 10"></polyline>
                    <line x1="12" y1="15" x2="12" y2="3"></line>
                </svg>
                        PDF
            </button>
            <button type="button" class="btn danger-btn" id="clear-all-btn">
                <svg viewBox="0 0 24 24" width="16" height="16" stroke="currentColor
                    <polyline points="3 6 5 6 21 6"></polyline>
                    <path d="M19 6v14a2 2 0 0 1-2 2H7a2 2 0 0 1-2-2V6m3 0V4a2 2 0 0
                </svg>

            </button>
        </div>
    </div>
</aside>

<!-- Main Viewport Area with Rulers and Canvas -->
<section class="composer-viewport">
    <div class="viewport-toolbar">
        <div class="viewport-status">
            <span id="paper-info-label">A4      ) ×      <span/>(
            <span id="cursor-pos-label">   :  ,     <span/>
        </div>
        <div class="zoom-controls">
            <button type="button" class="zoom-btn" id="zoom-out-btn" title="  <butto
            <span class="zoom-level" id="zoom-level-text">100%</span>
            <button type="button" class="zoom-btn" id="zoom-in-btn" title="  <button
            <button type="button" class="zoom-btn" id="zoom-fit-btn" title="      "
```

```
            </div>
        </div>

        <div class="composer-canvas-area" id="composer-canvas-area">
            <!-- Top-Left Corner -->
            <div class="ruler-corner">mm</div>

            <!-- Horizontal Ruler (Top) -->
            <div class="ruler-h-wrapper" id="ruler-h-wrapper">
                <canvas class="ruler-h" id="ruler-h"></canvas>
            </div>

            <!-- Vertical Ruler (Left) -->
            <div class="ruler-v-wrapper" id="ruler-v-wrapper">
                <canvas class="ruler-v" id="ruler-v"></canvas>
            </div>

            <!-- Paper Canvas Workspace -->
            <div class="paper-canvas-wrapper" id="paper-canvas-wrapper">
                <canvas id="paper-canvas"></canvas>
                <div id="size-badge" class="size-badge" style="display: none;"></div>
            </div>
        </div>
    </section>
</main>

<script src="composer.js"></script>
</body>
</html>
```

فایل composer.js:

```
/**
 * Layout Composer Script
 * Drag-and-drop printable document layout composer
 * Offline Chrome Extension Manifest V3
 */

document.addEventListener('DOMContentLoaded', () => {
    // Paper dimensions in millimeters (width x height in portrait)
    const PAPER_SIZES = {
        'a3': { w: 297, h: 420, label: 'A3' },
        'a4': { w: 210, h: 297, label: 'A4' },
        'a5': { w: 148, h: 210, label: 'A5' }
    };
```

```
const MM_PER_CM = 10;
const MARGIN = 10; // mm
const SNAP_THRESHOLD = 2.5; // mm
const GRID_SIZE = 10; // mm (1cm)
const GRID_MINOR = 5; // mm
const HANDLE_SIZE = 8; // Screen pixels for corner handles

// UI Elements
const paperSizeRadios = document.querySelectorAll('input[name="paper-size"]');
const orientationRadios = document.querySelectorAll('input[name="orientation"]');
const toggleGridCheckbox = document.getElementById('toggle-grid');
const toggleGuidelinesCheckbox = document.getElementById('toggle-guidelines');
const imageFileInput = document.getElementById('image-file-input');
const addImageBtn = document.getElementById('add-image-btn');
const imageListContainer = document.getElementById('image-list');
const imageListEmpty = document.getElementById('image-list-empty');
const autoLayoutBtn = document.getElementById('auto-layout-btn');
const exportPdfBtn = document.getElementById('export-pdf-btn');
const clearAllBtn = document.getElementById('clear-all-btn');

const zoomOutBtn = document.getElementById('zoom-out-btn');
const zoomInBtn = document.getElementById('zoom-in-btn');
const zoomFitBtn = document.getElementById('zoom-fit-btn');
const zoomLevelText = document.getElementById('zoom-level-text');
const paperInfoLabel = document.getElementById('paper-info-label');
const cursorPosLabel = document.getElementById('cursor-pos-label');

const paperCanvasWrapper = document.getElementById('paper-canvas-wrapper');
const paperCanvas = document.getElementById('paper-canvas');
const paperCtx = paperCanvas.getContext('2d');

const rulerHCanvas = document.getElementById('ruler-h');
const rulerHCtx = rulerHCanvas.getContext('2d');
const rulerHWrapper = document.getElementById('ruler-h-wrapper');

const rulerVCanvas = document.getElementById('ruler-v');
const rulerVCtx = rulerVCanvas.getContext('2d');
const rulerVWrapper = document.getElementById('ruler-v-wrapper');

const sizeBadge = document.getElementById('size-badge');

// Selected Image Controls
const selectedImageGroup = document.getElementById('selected-image-group');
const imgWidthInput = document.getElementById('img-width-input');
const imgHeightInput = document.getElementById('img-height-input');
```

```
const imgPosXInput = document.getElementById('img-posx-input');
const imgPosYInput = document.getElementById('img-posy-input');
const lockAspectRatioCheckbox = document.getElementById('lock-aspect-ratio');
const matchHeightBtn = document.getElementById('match-height-btn');
const matchWidthBtn = document.getElementById('match-width-btn');
const composerRemoveBgBtn = document.getElementById('composer-remove-bg-btn');
const composerToleranceContainer = document.getElementById('composer-tolerance-containe
const composerBgTol = document.getElementById('composer-bg-tol');
const composerBgTolVal = document.getElementById('composer-bg-tol-val');
const rotateImageBtn = document.getElementById('rotate-image-btn');
const bringToFrontBtn = document.getElementById('bring-to-front-btn');
const sendToBackBtn = document.getElementById('send-to-back-btn');

// Crop Tool Controls
const toggleCropBtn = document.getElementById('toggle-crop-btn');
const cropPanel = document.getElementById('crop-panel');
const cropLeftSlider = document.getElementById('crop-left-slider');
const cropLeftVal = document.getElementById('crop-left-val');
const cropRightSlider = document.getElementById('crop-right-slider');
const cropRightVal = document.getElementById('crop-right-val');
const cropTopSlider = document.getElementById('crop-top-slider');
const cropTopVal = document.getElementById('crop-top-val');
const cropBottomSlider = document.getElementById('crop-bottom-slider');
const cropBottomVal = document.getElementById('crop-bottom-val');
const cropHalfRight = document.getElementById('crop-half-right');
const cropHalfLeft = document.getElementById('crop-half-left');
const cropHalfTop = document.getElementById('crop-half-top');
const cropHalfBottom = document.getElementById('crop-half-bottom');
const applyCropBtn = document.getElementById('apply-crop-btn');
const resetCropBtn = document.getElementById('reset-crop-btn');

// State
let paperSize = 'a4';
let orientation = 'portrait'; // or 'landscape'
let showGrid = false;
let showGuidelines = true;
let images = []; // Array of { id, img, x, y, width, height, naturalWidth, naturalHeight
let selectedImageId = null;
let scale = 1; // zoom multiplier (0.5 to 3.0)
let basePxPerMm = 2.5; // Calculated to fit paper nicely into viewport

// Interaction State
let isDragging = false;
let isResizing = false;
let dragOffset = { x: 0, y: 0 }; // in mm
let resizeHandle = null; // 'tl', 'tr', 'bl', 'br'
```

```
let resizeStart = { x: 0, y: 0, w: 0, h: 0, mouseX: 0, mouseY: 0 };
let guidelines = []; // Array of { type: 'h'|'v', pos: mm }
let nextImageId = 1;

// Core dimensions helper
function getPaperDimensions() {
    const base = PAPER_SIZES[paperSize] || PAPER_SIZES['a4'];
    if (orientation === 'landscape') {
        return { w: base.h, h: base.w, label: base.label };
    }
    return { w: base.w, h: base.h, label: base.label };
}

// Coordinate conversions between Millimeters and Screen Canvas Pixels
function mmToPixel(mm) {
    return mm * basePxPerMm * scale;
}

function pixelToMm(px) {
    return px / (basePxPerMm * scale);
}

// Calculate base px per mm so the paper fits comfortably in wrapper with padding
function updateBaseScale() {
    const dims = getPaperDimensions();
    const wrapperW = paperCanvasWrapper.clientWidth || 800;
    const wrapperH = paperCanvasWrapper.clientHeight || 600;
    const padding = 60; // total padding in px

    const availW = Math.max(100, wrapperW - padding);
    const availH = Math.max(100, wrapperH - padding);

    const scaleW = availW / dims.w;
    const scaleH = availH / dims.h;

    basePxPerMm = Math.min(scaleW, scaleH);
}

// Resize canvases to match container & paper dimensions with devicePixelRatio
function resizeCanvases() {
    const dpr = window.devicePixelRatio || 1;
    const dims = getPaperDimensions();

    const canvasWidthCss = Math.round(mmToPixel(dims.w));
    const canvasHeightCss = Math.round(mmToPixel(dims.h));
```

```
    // Paper canvas
    paperCanvas.width = canvasWidthCss * dpr;
    paperCanvas.height = canvasHeightCss * dpr;
    paperCanvas.style.width = canvasWidthCss + 'px';
    paperCanvas.style.height = canvasHeightCss + 'px';
    paperCtx.setTransform(dpr, 0, 0, dpr, 0, 0);

    // Horizontal Ruler canvas
    const rHW = rulerHWrapper.clientWidth || 800;
    const rHH = rulerHWrapper.clientHeight || 25;
    rulerHCanvas.width = rHW * dpr;
    rulerHCanvas.height = rHH * dpr;
    rulerHCanvas.style.width = rHW + 'px';
    rulerHCanvas.style.height = rHH + 'px';
    rulerHCtx.setTransform(dpr, 0, 0, dpr, 0, 0);

    // Vertical Ruler canvas
    const rVW = rulerVWrapper.clientWidth || 25;
    const rVH = rulerVWrapper.clientHeight || 600;
    rulerVCanvas.width = rVW * dpr;
    rulerVCanvas.height = rVH * dpr;
    rulerVCanvas.style.width = rVW + 'px';
    rulerVCanvas.style.height = rVH + 'px';
    rulerVCtx.setTransform(dpr, 0, 0, dpr, 0, 0);

    updateLabels();
}

// Update status labels
function updateLabels() {
    const dims = getPaperDimensions();
    const orientText = orientation === 'portrait' ? '   ' : '  ';
    paperInfoLabel.textContent = `${dims.label} ${orientText} (${dims.w} × ${dims.h}
    zoomLevelText.textContent = `${Math.round(scale * 100)}%`;
}

// Main render function
function render() {
    renderPaper();
    renderGrid();
    renderImages();
    renderSelection();
    renderGuidelines();
    renderRulers();
}
```

```
// Render Paper background
function renderPaper() {
    const dims = getPaperDimensions();
    const w = mmToPixel(dims.w);
    const h = mmToPixel(dims.h);

    paperCtx.clearRect(0, 0, w, h);
    paperCtx.fillStyle = '#ffffff';
    paperCtx.fillRect(0, 0, w, h);
}

// Render Grid
function renderGrid() {
    if (!showGrid) return;

    const dims = getPaperDimensions();
    const w = mmToPixel(dims.w);
    const h = mmToPixel(dims.h);

    paperCtx.save();
    paperCtx.lineWidth = 0.5;

    // Minor grid (every 5mm)
    paperCtx.strokeStyle = 'rgba(215, 225, 235, 0.7)';
    paperCtx.beginPath();
    for (let x = GRID_MINOR; x < dims.w; x += GRID_MINOR) {
        if (x % GRID_SIZE === 0) continue;
        const px = Math.round(mmToPixel(x)) + 0.5;
        paperCtx.moveTo(px, 0);
        paperCtx.lineTo(px, h);
    }
    for (let y = GRID_MINOR; y < dims.h; y += GRID_MINOR) {
        if (y % GRID_SIZE === 0) continue;
        const py = Math.round(mmToPixel(y)) + 0.5;
        paperCtx.moveTo(0, py);
        paperCtx.lineTo(w, py);
    }
    paperCtx.stroke();

    // Major grid (every 10mm / 1cm)
    paperCtx.strokeStyle = 'rgba(170, 195, 220, 0.85)';
    paperCtx.beginPath();
    for (let x = GRID_SIZE; x < dims.w; x += GRID_SIZE) {
        const px = Math.round(mmToPixel(x)) + 0.5;
        paperCtx.moveTo(px, 0);
        paperCtx.lineTo(px, h);
```

```
    }
    for (let y = GRID_SIZE; y < dims.h; y += GRID_SIZE) {
        const py = Math.round(mmToPixel(y)) + 0.5;
        paperCtx.moveTo(0, py);
        paperCtx.lineTo(w, py);
    }
    paperCtx.stroke();

    paperCtx.restore();
}

// Render Images
function renderImages() {
    for (const item of images) {
        const px = mmToPixel(item.x);
        const py = mmToPixel(item.y);
        const pw = mmToPixel(item.width);
        const ph = mmToPixel(item.height);

        paperCtx.save();
        try {
            paperCtx.drawImage(item.img, px, py, pw, ph);
        } catch (err) {
            console.error('Error drawing image:', err);
        }

        // Light border around image
        paperCtx.strokeStyle = 'rgba(0, 0, 0, 0.15)';
        paperCtx.lineWidth = 1;
        paperCtx.strokeRect(px, py, pw, ph);
        paperCtx.restore();
    }
}

// Render Selection Frame and Resize Handles
function renderSelection() {
    const selected = images.find(img => img.id === selectedImageId);
    if (!selected) return;

    const px = mmToPixel(selected.x);
    const py = mmToPixel(selected.y);
    const pw = mmToPixel(selected.width);
    const ph = mmToPixel(selected.height);

    paperCtx.save();
```

```
    // Selection border
    paperCtx.strokeStyle = '#0078d7';
    paperCtx.lineWidth = 1.5;
    paperCtx.setLineDash([4, 3]);
    paperCtx.strokeRect(px, py, pw, ph);
    paperCtx.setLineDash([]);

    // Corner handles
    const handles = getHandleCoordinates(px, py, pw, ph);
    paperCtx.fillStyle = '#ffffff';
    paperCtx.strokeStyle = '#0078d7';
    paperCtx.lineWidth = 2;

    const hs = HANDLE_SIZE;
    for (const key in handles) {
        const hCoord = handles[key];
        paperCtx.fillRect(hCoord.x - hs / 2, hCoord.y - hs / 2, hs, hs);
        paperCtx.strokeRect(hCoord.x - hs / 2, hCoord.y - hs / 2, hs, hs);
    }
    // Dimension badge attached to bottom of selection
    const cmW = (selected.width / 10).toFixed(1);
    const cmH = (selected.height / 10).toFixed(1);
    const tagText = `${cmW} × ${cmH} cm`;

    paperCtx.font = 'bold 11px Tahoma, sans-serif';
    const textMetrics = paperCtx.measureText(tagText);
    const tagW = textMetrics.width + 12;
    const tagH = 18;
    const tagX = px + (pw - tagW) / 2;
    const tagY = py + ph + 8;

    paperCtx.fillStyle = 'rgba(0, 120, 215, 0.95)';
    paperCtx.fillRect(tagX, tagY, tagW, tagH);
    paperCtx.fillStyle = '#ffffff';
    paperCtx.textAlign = 'center';
    paperCtx.textBaseline = 'middle';
    paperCtx.fillText(tagText, tagX + tagW / 2, tagY + tagH / 2);

    paperCtx.restore();
}

// Helper: Corner handles coordinates
function getHandleCoordinates(px, py, pw, ph) {
    return {
        tl: { x: px, y: py },
        tr: { x: px + pw, y: py },
```

```
        br: { x: px + pw, y: py + ph },
        bl: { x: px, y: py + ph }
    };
}

// Render Guidelines
function renderGuidelines() {
    if (!showGuidelines || guidelines.length === 0) return;

    const dims = getPaperDimensions();
    const maxW = mmToPixel(dims.w);
    const maxH = mmToPixel(dims.h);

    paperCtx.save();
    paperCtx.lineWidth = 1;
    paperCtx.setLineDash([4, 4]);

    for (const g of guidelines) {
        paperCtx.strokeStyle = g.color || '#0078d7';
        paperCtx.beginPath();
        if (g.type === 'v') {
            const px = Math.round(mmToPixel(g.pos)) + 0.5;
            paperCtx.moveTo(px, 0);
            paperCtx.lineTo(px, maxH);
        } else if (g.type === 'h') {
            const py = Math.round(mmToPixel(g.pos)) + 0.5;
            paperCtx.moveTo(0, py);
            paperCtx.lineTo(maxW, py);
        }
        paperCtx.stroke();
    }

    paperCtx.restore();
}

// Render Rulers (top horizontal and left vertical)
function renderRulers() {
    const dims = getPaperDimensions();

    // Calculate offset of paper canvas relative to ruler wrappers
    const paperRect = paperCanvas.getBoundingClientRect();
    const rHRect = rulerHWrapper.getBoundingClientRect();
    const rVRect = rulerVWrapper.getBoundingClientRect();

    const paperOffsetLeft = paperRect.left - rHRect.left;
    const paperOffsetTop = paperRect.top - rVRect.top;
```

```
// 1. Horizontal Ruler
const rHW = rulerHCanvas.width / (window.devicePixelRatio || 1);
const rHH = rulerHCanvas.height / (window.devicePixelRatio || 1);
rulerHCtx.clearRect(0, 0, rHW, rHH);

rulerHCtx.fillStyle = '#f5f5f5';
rulerHCtx.fillRect(0, 0, rHW, rHH);

rulerHCtx.fillStyle = '#333333';
rulerHCtx.strokeStyle = '#888888';
rulerHCtx.font = '9px Tahoma, sans-serif';
rulerHCtx.textAlign = 'center';
rulerHCtx.textBaseline = 'top';

// Ruler ticks along paper width
for (let mm = 0; mm <= dims.w; mm += 1) {
    const screenX = paperOffsetLeft + mmToPixel(mm);
    if (screenX < -20 || screenX > rHW + 20) continue;

    const isMajor = mm % 10 === 0;
    const isMedium = mm % 5 === 0;

    if (isMajor) {
        rulerHCtx.beginPath();
        rulerHCtx.moveTo(screenX, rHH - 12);
        rulerHCtx.lineTo(screenX, rHH);
        rulerHCtx.stroke();

        const cm = mm / 10;
        rulerHCtx.fillText(cm.toString(), screenX, 3);
    } else if (isMedium) {
        rulerHCtx.beginPath();
        rulerHCtx.moveTo(screenX, rHH - 7);
        rulerHCtx.lineTo(screenX, rHH);
        rulerHCtx.stroke();
    } else if (scale >= 1.2) {
        // Minor 1mm ticks if zoomed in
        rulerHCtx.beginPath();
        rulerHCtx.moveTo(screenX, rHH - 4);
        rulerHCtx.lineTo(screenX, rHH);
        rulerHCtx.stroke();
    }
}

// 2. Vertical Ruler
```

```
    const rVW = rulerVCanvas.width / (window.devicePixelRatio || 1);
    const rVH = rulerVCanvas.height / (window.devicePixelRatio || 1);
    rulerVCtx.clearRect(0, 0, rVW, rVH);

    rulerVCtx.fillStyle = '#f5f5f5';
    rulerVCtx.fillRect(0, 0, rVW, rVH);

    rulerVCtx.fillStyle = '#333333';
    rulerVCtx.strokeStyle = '#888888';
    rulerVCtx.font = '9px Tahoma, sans-serif';
    rulerVCtx.textAlign = 'right';
    rulerVCtx.textBaseline = 'middle';

    for (let mm = 0; mm <= dims.h; mm += 1) {
        const screenY = paperOffsetTop + mmToPixel(mm);
        if (screenY < -20 || screenY > rVH + 20) continue;

        const isMajor = mm % 10 === 0;
        const isMedium = mm % 5 === 0;

        if (isMajor) {
            rulerVCtx.beginPath();
            rulerVCtx.moveTo(rVW - 12, screenY);
            rulerVCtx.lineTo(rVW, screenY);
            rulerVCtx.stroke();

            const cm = mm / 10;
            rulerVCtx.fillText(cm.toString(), rVW - 13, screenY);
        } else if (isMedium) {
            rulerVCtx.beginPath();
            rulerVCtx.moveTo(rVW - 7, screenY);
            rulerVCtx.lineTo(rVW, screenY);
            rulerVCtx.stroke();
        } else if (scale >= 1.2) {
            // Minor 1mm ticks
            rulerVCtx.beginPath();
            rulerVCtx.moveTo(rVW - 4, screenY);
            rulerVCtx.lineTo(rVW, screenY);
            rulerVCtx.stroke();
        }
    }
}

// Image Management
function addImage(file) {
    if (!file || !file.type.startsWith('image/')) return;
```

```
const reader = new FileReader();
reader.onload = (e) => {
    const img = new Image();
    img.onload = () => {
        const dims = getPaperDimensions();
        const naturalW = img.naturalWidth || img.width;
        const naturalH = img.naturalHeight || img.height;
        const aspect = naturalW / naturalH;

        // Initial size: fit image to max 50% of paper width, maintaining aspect rat
        let initW = (dims.w - MARGIN * 2) * 0.5;
        let initH = initW / aspect;

        // Check if height exceeds 50% of paper height
        const maxH = (dims.h - MARGIN * 2) * 0.5;
        if (initH > maxH) {
            initH = maxH;
            initW = initH * aspect;
        }

        // Initial position: center-ish or cascaded
        const offset = (images.length % 5) * 8;
        let initX = MARGIN + offset;
        let initY = MARGIN + offset;

        if (initX + initW > dims.w - MARGIN) initX = MARGIN;
        if (initY + initH > dims.h - MARGIN) initY = MARGIN;

        const imageItem = {
            id: 'img_' + (nextImageId++),
            img: img,
            originalImg: img,
            name: file.name || `    ${images.length + 1}`,
            x: Math.round(initX * 10) / 10,
            y: Math.round(initY * 10) / 10,
            width: Math.round(initW * 10) / 10,
            height: Math.round(initH * 10) / 10,
            naturalWidth: naturalW,
            naturalHeight: naturalH
        };

        images.push(imageItem);
        selectedImageId = imageItem.id;

        updateImageList();
```

```
            render();
        };
        img.src = e.target.result;
    };
    reader.readAsDataURL(file);
}

function removeImage(id) {
    images = images.filter(img => img.id !== id);
    if (selectedImageId === id) {
        selectedImageId = images.length > 0 ? images[images.length - 1].id : null;
    }
    updateImageList();
    render();
}

function updateImageList() {
    if (images.length === 0) {
        imageListEmpty.style.display = 'block';
        imageListContainer.innerHTML = '';
        imageListContainer.appendChild(imageListEmpty);
        return;
    }

    imageListEmpty.style.display = 'none';
    imageListContainer.innerHTML = '';

    images.forEach(item => {
        const row = document.createElement('div');
        row.className = `image-item ${item.id === selectedImageId ? 'active' : ''}`;
        row.dataset.id = item.id;

        const thumb = document.createElement('img');
        thumb.className = 'image-item-thumb';
        thumb.src = item.img.src;

        const info = document.createElement('div');
        info.className = 'image-item-info';

        const title = document.createElement('span');
        title.className = 'image-item-title';
        title.textContent = item.name;

        const dims = document.createElement('span');
        dims.className = 'image-item-dims';
        dims.textContent = `${(item.width / 10).toFixed(1)} × ${(item.height / 10).toFix
```

```
        info.appendChild(title);
        info.appendChild(dims);

        const actions = document.createElement('div');
        actions.className = 'image-item-actions';

        const delBtn = document.createElement('button');
        delBtn.className = 'image-item-delete';
        delBtn.title = '      ';
        delBtn.innerHTML = `
            <svg viewBox="0 0 24 24" width="14" height="14" stroke="currentColor" stroke
                <line x1="18" y1="6" x2="6" y2="18"></line>
                <line x1="6" y1="6" x2="18" y2="18"></line>
            </svg>
        `;
        delBtn.addEventListener('click', (e) => {
            e.stopPropagation();
            removeImage(item.id);
        });

        actions.appendChild(delBtn);

        row.appendChild(thumb);
        row.appendChild(info);
        row.appendChild(actions);

        row.addEventListener('click', () => {
            selectedImageId = item.id;
            updateImageList();
            render();
        });

        imageListContainer.appendChild(row);
    });
    updateSelectedImageInputs();
}

function updateSelectedImageInputs() {
    if (!selectedImageGroup) return;
    const selected = images.find(img => img.id === selectedImageId);
    if (!selected) {
        selectedImageGroup.style.display = 'none';
        return;
    }
```

```
    selectedImageGroup.style.display = 'block';
    if (imgWidthInput && document.activeElement !== imgWidthInput) {
        imgWidthInput.value = (selected.width / 10).toFixed(1);
    }
    if (imgHeightInput && document.activeElement !== imgHeightInput) {
        imgHeightInput.value = (selected.height / 10).toFixed(1);
    }
    if (imgPosXInput && document.activeElement !== imgPosXInput) {
        imgPosXInput.value = (selected.x / 10).toFixed(1);
    }
    if (imgPosYInput && document.activeElement !== imgPosYInput) {
        imgPosYInput.value = (selected.y / 10).toFixed(1);
    }
}

// Hit Testing and Interaction
function getCanvasCoordinates(e) {
    const rect = paperCanvas.getBoundingClientRect();
    const clientX = e.clientX;
    const clientY = e.clientY;

    const screenX = clientX - rect.left;
    const screenY = clientY - rect.top;

    const mmX = pixelToMm(screenX);
    const mmY = pixelToMm(screenY);

    return { screenX, screenY, mmX, mmY };
}

function findImageAtPoint(mmX, mmY) {
    // Iterate backwards so top-most image is selected
    for (let i = images.length - 1; i >= 0; i--) {
        const item = images[i];
        if (
            mmX >= item.x &&
            mmX <= item.x + item.width &&
            mmY >= item.y &&
            mmY <= item.y + item.height
        ) {
            return item;
        }
    }
    return null;
}
```

```
function findResizeHandle(img, screenX, screenY) {
    if (!img) return null;

    const px = mmToPixel(img.x);
    const py = mmToPixel(img.y);
    const pw = mmToPixel(img.width);
    const ph = mmToPixel(img.height);

    const handles = getHandleCoordinates(px, py, pw, ph);
    const hitDistance = 10; // pixels

    for (const [key, coord] of Object.entries(handles)) {
        const dx = Math.abs(screenX - coord.x);
        const dy = Math.abs(screenY - coord.y);
        if (dx <= hitDistance && dy <= hitDistance) {
            return key;
        }
    }
    return null;
}

function detectAlignmentGuides(movingImg) {
    const guides = [];
    let snappedX = movingImg.x;
    let snappedY = movingImg.y;
    let snappedRight = movingImg.x + movingImg.width;
    let snappedBottom = movingImg.y + movingImg.height;

    const paperDims = getPaperDimensions();

    // Targets to align against: Paper Margins & Centers, and other images
    const verticalTargets = [
        MARGIN,
        paperDims.w / 2,
        paperDims.w - MARGIN
    ];

    const horizontalTargets = [
        MARGIN,
        paperDims.h / 2,
        paperDims.h - MARGIN
    ];

    images.forEach(other => {
        if (other.id === movingImg.id) return;
        verticalTargets.push(other.x);
```

```
    verticalTargets.push(other.x + other.width / 2);
    verticalTargets.push(other.x + other.width);

    horizontalTargets.push(other.y);
    horizontalTargets.push(other.y + other.height / 2);
    horizontalTargets.push(other.y + other.height);
});

const movingCenterX = movingImg.x + movingImg.width / 2;
const movingRight = movingImg.x + movingImg.width;

// Check vertical alignments (X coordinates)
let vSnapped = false;
for (const target of verticalTargets) {
    // Left edge
    if (Math.abs(movingImg.x - target) <= SNAP_THRESHOLD) {
        snappedX = target;
        guides.push({ type: 'v', pos: target, color: '#0078d7' });
        vSnapped = true;
        break;
    }
    // Right edge
    if (Math.abs(movingRight - target) <= SNAP_THRESHOLD) {
        snappedX = target - movingImg.width;
        guides.push({ type: 'v', pos: target, color: '#0078d7' });
        vSnapped = true;
        break;
    }
    // Center
    if (Math.abs(movingCenterX - target) <= SNAP_THRESHOLD) {
        snappedX = target - movingImg.width / 2;
        guides.push({ type: 'v', pos: target, color: '#e81123' });
        vSnapped = true;
        break;
    }
}

// Check horizontal alignments (Y coordinates)
const movingCenterY = movingImg.y + movingImg.height / 2;
const movingBottom = movingImg.y + movingImg.height;

for (const target of horizontalTargets) {
    // Top edge
    if (Math.abs(movingImg.y - target) <= SNAP_THRESHOLD) {
        snappedY = target;
        guides.push({ type: 'h', pos: target, color: '#0078d7' });
```

```
            break;
        }
        // Bottom edge
        if (Math.abs(movingBottom - target) <= SNAP_THRESHOLD) {
            snappedY = target - movingImg.height;
            guides.push({ type: 'h', pos: target, color: '#0078d7' });
            break;
        }
        // Center
        if (Math.abs(movingCenterY - target) <= SNAP_THRESHOLD) {
            snappedY = target - movingImg.height / 2;
            guides.push({ type: 'h', pos: target, color: '#e81123' });
            break;
        }
    }

    return { guides, snappedX, snappedY };
}

// Show floating size/position badge
function showBadge(text, screenX, screenY) {
    const wrapperRect = paperCanvasWrapper.getBoundingClientRect();
    const canvasRect = paperCanvas.getBoundingClientRect();

    const badgeX = canvasRect.left - wrapperRect.left + screenX;
    const badgeY = canvasRect.top - wrapperRect.top + screenY;

    sizeBadge.style.left = badgeX + 'px';
    sizeBadge.style.top = badgeY + 'px';
    sizeBadge.textContent = text;
    sizeBadge.style.display = 'block';
}

function hideBadge() {
    sizeBadge.style.display = 'none';
}

// Mouse Event Handlers
function handleMouseDown(e) {
    if (e.button !== 0) return; // Only left button

    const coords = getCanvasCoordinates(e);
    const selected = images.find(img => img.id === selectedImageId);

    // Check handle click on currently selected image
    if (selected) {
```

```
            const handle = findResizeHandle(selected, coords.screenX, coords.screenY);
            if (handle) {
                isResizing = true;
                resizeHandle = handle;
                resizeStart = {
                    x: selected.x,
                    y: selected.y,
                    w: selected.width,
                    h: selected.height,
                    aspect: selected.naturalWidth / selected.naturalHeight,
                    mmX: coords.mmX,
                    mmY: coords.mmY
                };
                return;
            }
        }

        // Check clicking on an image
        const clickedImg = findImageAtPoint(coords.mmX, coords.mmY);
        if (clickedImg) {
            selectedImageId = clickedImg.id;
            // Bring clicked image to front
            images = images.filter(img => img.id !== clickedImg.id).concat([clickedImg]);

            isDragging = true;
            dragOffset = {
                x: coords.mmX - clickedImg.x,
                y: coords.mmY - clickedImg.y
            };

            updateImageList();
            render();
            return;
        }

        // Clicked on blank paper canvas: deselect
        selectedImageId = null;
        updateImageList();
        render();
    }

    function handleMouseMove(e) {
        const coords = getCanvasCoordinates(e);

        // Update cursor position display in toolbar (in cm)
        const cmX = (coords.mmX / 10).toFixed(1);
```

```
const cmY = (coords.mmY / 10).toFixed(1);
cursorPosLabel.textContent = `:    ${cmX}, ${cmY}      `;

const selected = images.find(img => img.id === selectedImageId);

// If dragging an image
if (isDragging && selected) {
    const paperDims = getPaperDimensions();
    let newX = coords.mmX - dragOffset.x;
    let newY = coords.mmY - dragOffset.y;

    // Grid snap if enabled
    if (showGrid) {
        newX = Math.round(newX / GRID_MINOR) * GRID_MINOR;
        newY = Math.round(newY / GRID_MINOR) * GRID_MINOR;
    }

    // Magnetic Alignment Guidelines snap if enabled
    if (showGuidelines) {
        const align = detectAlignmentGuides({
            ...selected,
            x: newX,
            y: newY
        });
        guidelines = align.guides;
        newX = align.snappedX;
        newY = align.snappedY;
    } else {
        guidelines = [];
    }

    // Allow overhang past paper boundaries (keep at least 10mm on paper so it is no
    const minVisible = 10; // mm
    newX = Math.max(-selected.width + minVisible, Math.min(paperDims.w - minVisible,
    newY = Math.max(-selected.height + minVisible, Math.min(paperDims.h - minVisible

    selected.x = Math.round(newX * 10) / 10;
    selected.y = Math.round(newY * 10) / 10;
    updateSelectedImageInputs();

    showBadge(
        `X: ${(selected.x / 10).toFixed(1)} cm | Y: ${(selected.y / 10).toFixed(1)}
        mmToPixel(selected.x + selected.width / 2),
        mmToPixel(selected.y)
    );
```

```
    render();
    return;
}

// If resizing an image
if (isResizing && selected) {
    const deltaX = coords.mmX - resizeStart.mmX;
    const deltaY = coords.mmY - resizeStart.mmY;
    const aspect = resizeStart.aspect;
    const keepAspect = !e.shiftKey; // Default keep aspect ratio unless Shift is pre

    let newX = resizeStart.x;
    let newY = resizeStart.y;
    let newW = resizeStart.w;
    let newH = resizeStart.h;

    const minSize = 10; // 10mm minimum

    if (resizeHandle === 'br') {
        newW = Math.max(minSize, resizeStart.w + deltaX);
        newH = keepAspect ? newW / aspect : Math.max(minSize, resizeStart.h + deltaY
    } else if (resizeHandle === 'bl') {
        newW = Math.max(minSize, resizeStart.w - deltaX);
        newX = resizeStart.x + (resizeStart.w - newW);
        newH = keepAspect ? newW / aspect : Math.max(minSize, resizeStart.h + deltaY
    } else if (resizeHandle === 'tr') {
        newW = Math.max(minSize, resizeStart.w + deltaX);
        newH = keepAspect ? newW / aspect : Math.max(minSize, resizeStart.h - deltaY
        newY = resizeStart.y + (resizeStart.h - newH);
    } else if (resizeHandle === 'tl') {
        newW = Math.max(minSize, resizeStart.w - deltaX);
        newX = resizeStart.x + (resizeStart.w - newW);
        newH = keepAspect ? newW / aspect : Math.max(minSize, resizeStart.h - deltaY
        newY = resizeStart.y + (resizeStart.h - newH);
    }

    // Grid snap for width/height
    if (showGrid) {
        newW = Math.round(newW / GRID_MINOR) * GRID_MINOR;
        newH = Math.round(newH / GRID_MINOR) * GRID_MINOR;
    }

    selected.x = Math.round(newX * 10) / 10;
    selected.y = Math.round(newY * 10) / 10;
    selected.width = Math.round(newW * 10) / 10;
    selected.height = Math.round(newH * 10) / 10;
```

```
            showBadge(
                `${(selected.width / 10).toFixed(1)} × ${(selected.height / 10).toFixed(1)}
                mmToPixel(selected.x + selected.width / 2),
                mmToPixel(selected.y)
            );

            render();
            return;
        }

        // Cursor style update when hovering
        if (selected) {
            const handle = findResizeHandle(selected, coords.screenX, coords.screenY);
            if (handle) {
                if (handle === 'tl' || handle === 'br') paperCanvas.style.cursor = 'nwse-res
                else if (handle === 'tr' || handle === 'bl') paperCanvas.style.cursor = 'nes
                return;
            }
        }

        const hoveredImg = findImageAtPoint(coords.mmX, coords.mmY);
        if (hoveredImg) {
            paperCanvas.style.cursor = 'move';
        } else {
            paperCanvas.style.cursor = 'default';
        }
    }

    function handleMouseUp() {
        if (isDragging || isResizing) {
            isDragging = false;
            isResizing = false;
            resizeHandle = null;
            guidelines = [];
            hideBadge();
            updateImageList();
            render();
        }
    }

    // Mouse wheel zoom
    function handleWheel(e) {
        e.preventDefault();
        const delta = e.deltaY < 0 ? 0.1 : -0.1;
        const newScale = Math.min(3.0, Math.max(0.5, scale + delta));
```

```
    if (newScale !== scale) {
        scale = newScale;
        resizeCanvases();
        render();
    }
}

// Auto-Layout Algorithm
function autoLayout() {
    if (images.length === 0) return;

    const dims = getPaperDimensions();
    const gap = 5; // 5mm gap between images
    const availWidth = dims.w - MARGIN * 2;
    const availHeight = dims.h - MARGIN * 2;

    // Sort images by natural height descending (tallest first)
    const sorted = [...images].sort((a, b) => b.naturalHeight - a.naturalHeight);

    // Check if images are too large to fit in rows; scale if needed
    let maxRowW = availWidth * 0.45;
    sorted.forEach(item => {
        const aspect = item.naturalWidth / item.naturalHeight;
        let targetW = Math.min(maxRowW, item.width);
        let targetH = targetW / aspect;
        item.width = Math.round(targetW * 10) / 10;
        item.height = Math.round(targetH * 10) / 10;
    });

    let curX = MARGIN;
    let curY = MARGIN;
    let rowHeight = 0;

    for (const item of sorted) {
        // If doesn't fit on current row horizontally, start a new row
        if (curX + item.width > dims.w - MARGIN && curX > MARGIN) {
            curX = MARGIN;
            curY += rowHeight + gap;
            rowHeight = 0;
        }

        // Check if it fits on page vertically
        if (curY + item.height > dims.h - MARGIN) {
            // Scale down slightly to fit on page
            const overflowFactor = (dims.h - MARGIN - curY) / item.height;
            if (overflowFactor > 0.4) {
```

```
                const aspect = item.naturalWidth / item.naturalHeight;
                item.height = Math.max(15, (dims.h - MARGIN - curY));
                item.width = item.height * aspect;
            }
        }

        item.x = Math.round(curX * 10) / 10;
        item.y = Math.round(curY * 10) / 10;

        curX += item.width + gap;
        rowHeight = Math.max(rowHeight, item.height);
    }

    updateImageList();
    render();
}

// PDF Export
async function exportPDF() {
    if (images.length === 0) {
        alert('                          .  ';(
        return;
    }

    const jsPDF = window.jspdf ? window.jspdf.jsPDF : window.jsPDF;
    if (!jsPDF) {
        alert('        PDF          .  ';(
        return;
    }

    exportPdfBtn.disabled = true;
    const originalText = exportPdfBtn.innerHTML;
    exportPdfBtn.innerHTML = '        ...  ';

    try {
        const dims = getPaperDimensions();
        const doc = new jsPDF({
            orientation: orientation,
            unit: 'mm',
            format: [dims.w, dims.h]
        });

        for (const item of images) {
            // Calculate visible intersection with paper boundaries
            const visX = Math.max(0, item.x);
            const visY = Math.max(0, item.y);
```

```
const visRight = Math.min(dims.w, item.x + item.width);
const visBottom = Math.min(dims.h, item.y + item.height);
const visW = visRight - visX;
const visH = visBottom - visY;

if (visW <= 0.1 || visH <= 0.1) continue; // Completely outside paper

// If fully inside paper bounds
if (item.x >= 0 && item.y >= 0 && item.x + item.width <= dims.w && item.y +
    let dataUrl;
    if (item.img.src && item.img.src.startsWith('data:image/png')) {
        dataUrl = item.img.src;
    } else {
        const tempCanvas = document.createElement('canvas');
        tempCanvas.width = item.naturalWidth;
        tempCanvas.height = item.naturalHeight;
        const tempCtx = tempCanvas.getContext('2d');
        tempCtx.drawImage(item.img, 0, 0);
        dataUrl = tempCanvas.toDataURL('image/png');
    }
    const isPng = dataUrl.startsWith('data:image/png');
    doc.addImage(
        dataUrl,
        isPng ? 'PNG' : 'JPEG',
        item.x,
        item.y,
        item.width,
        item.height,
        undefined,
        'FAST'
    );
} else {
    // Partially outside paper: crop only the visible slice inside the paper
    const scaleX = item.naturalWidth / item.width;
    const scaleY = item.naturalHeight / item.height;

    const sx = Math.max(0, Math.round((visX - item.x) * scaleX));
    const sy = Math.max(0, Math.round((visY - item.y) * scaleY));
    const sw = Math.min(item.naturalWidth - sx, Math.round(visW * scaleX));
    const sh = Math.min(item.naturalHeight - sy, Math.round(visH * scaleY));

    if (sw <= 0 || sh <= 0) continue;

    const cropCanvas = document.createElement('canvas');
    cropCanvas.width = sw;
    cropCanvas.height = sh;
```

```
                const cropCtx = cropCanvas.getContext('2d');
                cropCtx.drawImage(item.img, sx, sy, sw, sh, 0, 0, sw, sh);

                const croppedDataUrl = cropCanvas.toDataURL('image/png');
                doc.addImage(
                    croppedDataUrl,
                    'PNG',
                    visX,
                    visY,
                    visW,
                    visH,
                    undefined,
                    'FAST'
                );
            }
        }

        const fileName = `layout_${paperSize}_${orientation}_${new Date().toISOString().
        doc.save(fileName);
    } catch (err) {
        console.error('Error generating PDF:', err);
        alert('                PDF   :   ' + err.message);
    } finally {
        exportPdfBtn.disabled = false;
        exportPdfBtn.innerHTML = originalText;
    }
}

// Event Listeners
paperSizeRadios.forEach(radio => {
    radio.addEventListener('change', (e) => {
        paperSize = e.target.value;
        updateBaseScale();
        resizeCanvases();
        render();
    });
});

orientationRadios.forEach(radio => {
    radio.addEventListener('change', (e) => {
        orientation = e.target.value;
        updateBaseScale();
        resizeCanvases();
        render();
    });
});
```

```
toggleGridCheckbox.addEventListener('change', (e) => {
    showGrid = e.target.checked;
    render();
});

toggleGuidelinesCheckbox.addEventListener('change', (e) => {
    showGuidelines = e.target.checked;
    render();
});

// File Input
addImageBtn.addEventListener('click', () => {
    imageFileInput.click();
});

imageFileInput.addEventListener('change', (e) => {
    const files = Array.from(e.target.files);
    files.forEach(file => addImage(file));
    imageFileInput.value = '';
});

// Auto Layout and Export buttons
autoLayoutBtn.addEventListener('click', autoLayout);
exportPdfBtn.addEventListener('click', exportPDF);

clearAllBtn.addEventListener('click', () => {
    if (images.length === 0) return;
    if (confirm('                      '(( {
        images = [];
        selectedImageId = null;
        updateImageList();
        render();
    }
});

// Zoom buttons
zoomInBtn.addEventListener('click', () => {
    scale = Math.min(3.0, scale + 0.15);
    resizeCanvases();
    render();
});

zoomOutBtn.addEventListener('click', () => {
    scale = Math.max(0.5, scale - 0.15);
    resizeCanvases();
```

```
    render();
});

zoomFitBtn.addEventListener('click', () => {
    scale = 1.0;
    updateBaseScale();
    resizeCanvases();
    render();
});

// Canvas Events
paperCanvas.addEventListener('mousedown', handleMouseDown);
window.addEventListener('mousemove', handleMouseMove);
window.addEventListener('mouseup', handleMouseUp);
paperCanvasWrapper.addEventListener('wheel', handleWheel, { passive: false });

// Keyboard Shortcuts (Delete, Arrow Keys)
window.addEventListener('keydown', (e) => {
    if (!selectedImageId) return;

    if (e.key === 'Delete' || e.key === 'Backspace') {
        // Ignore if active element is an input
        if (['INPUT', 'TEXTAREA'].includes(document.activeElement.tagName)) return;
        e.preventDefault();
        removeImage(selectedImageId);
        return;
    }

    const selected = images.find(img => img.id === selectedImageId);
    if (!selected) return;

    const step = e.shiftKey ? 5 : 1; // 5mm with Shift, 1mm without
    const paperDims = getPaperDimensions();

    const minVisible = 10;
    if (e.key === 'ArrowLeft') {
        selected.x = Math.max(-selected.width + minVisible, selected.x - step);
        e.preventDefault();
    } else if (e.key === 'ArrowRight') {
        selected.x = Math.min(paperDims.w - minVisible, selected.x + step);
        e.preventDefault();
    } else if (e.key === 'ArrowUp') {
        selected.y = Math.max(-selected.height + minVisible, selected.y - step);
        e.preventDefault();
    } else if (e.key === 'ArrowDown') {
        selected.y = Math.min(paperDims.h - minVisible, selected.y + step);
```

```
        e.preventDefault();
    }

    selected.x = Math.round(selected.x * 10) / 10;
    selected.y = Math.round(selected.y * 10) / 10;
    updateSelectedImageInputs();
    render();
});

// Window resize observer to keep canvas centered and responsive
window.addEventListener('resize', () => {
    updateBaseScale();
    resizeCanvases();
    render();
});

// --- Selected Image Controls Event Listeners ---
if (imgWidthInput) {
    imgWidthInput.addEventListener('input', () => {
        const selected = images.find(img => img.id === selectedImageId);
        if (!selected) return;
        const cm = parseFloat(imgWidthInput.value);
        if (isNaN(cm) || cm <= 0) return;
        const newW = cm * 10;
        const aspect = selected.naturalWidth / selected.naturalHeight;
        selected.width = Math.round(newW * 10) / 10;
        if (lockAspectRatioCheckbox && lockAspectRatioCheckbox.checked) {
            selected.height = Math.round((newW / aspect) * 10) / 10;
            if (imgHeightInput) imgHeightInput.value = (selected.height / 10).toFixed(1)
        }
        updateImageList();
        render();
    });
}

if (imgHeightInput) {
    imgHeightInput.addEventListener('input', () => {
        const selected = images.find(img => img.id === selectedImageId);
        if (!selected) return;
        const cm = parseFloat(imgHeightInput.value);
        if (isNaN(cm) || cm <= 0) return;
        const newH = cm * 10;
        const aspect = selected.naturalWidth / selected.naturalHeight;
        selected.height = Math.round(newH * 10) / 10;
        if (lockAspectRatioCheckbox && lockAspectRatioCheckbox.checked) {
            selected.width = Math.round((newH * aspect) * 10) / 10;
```

```
            if (imgWidthInput) imgWidthInput.value = (selected.width / 10).toFixed(1);
        }
        updateImageList();
        render();
    });
}

if (matchHeightBtn) {
    matchHeightBtn.addEventListener('click', () => {
        const selected = images.find(img => img.id === selectedImageId);
        if (!selected) return;
        const other = images.find(img => img.id !== selectedImageId);
        if (!other) {
            alert('                          .   ';(
            return;
        }
        const aspect = selected.naturalWidth / selected.naturalHeight;
        selected.height = other.height;
        if (lockAspectRatioCheckbox && lockAspectRatioCheckbox.checked) {
            selected.width = Math.round((selected.height * aspect) * 10) / 10;
        }
        updateSelectedImageInputs();
        updateImageList();
        render();
    });
}

if (matchWidthBtn) {
    matchWidthBtn.addEventListener('click', () => {
        const selected = images.find(img => img.id === selectedImageId);
        if (!selected) return;
        const other = images.find(img => img.id !== selectedImageId);
        if (!other) {
            alert('                          .   ';(
            return;
        }
        const aspect = selected.naturalWidth / selected.naturalHeight;
        selected.width = other.width;
        if (lockAspectRatioCheckbox && lockAspectRatioCheckbox.checked) {
            selected.height = Math.round((selected.width / aspect) * 10) / 10;
        }
        updateSelectedImageInputs();
        updateImageList();
        render();
    });
}
```

```
if (rotateImageBtn) {
    rotateImageBtn.addEventListener('click', () => {
        const selected = images.find(img => img.id === selectedImageId);
        if (!selected) return;

        const tempCanvas = document.createElement('canvas');
        tempCanvas.width = selected.naturalHeight;
        tempCanvas.height = selected.naturalWidth;
        const ctx = tempCanvas.getContext('2d');
        ctx.translate(tempCanvas.width / 2, tempCanvas.height / 2);
        ctx.rotate(90 * Math.PI / 180);
        ctx.drawImage(selected.img, -selected.naturalWidth / 2, -selected.naturalHeight

        const newImg = new Image();
        newImg.onload = () => {
            selected.img = newImg;
            selected.naturalWidth = tempCanvas.width;
            selected.naturalHeight = tempCanvas.height;
            const oldW = selected.width;
            selected.width = selected.height;
            selected.height = oldW;
            updateSelectedImageInputs();
            updateImageList();
            render();
        };
        newImg.src = tempCanvas.toDataURL('image/png');
    });
}

if (bringToFrontBtn) {
    bringToFrontBtn.addEventListener('click', () => {
        if (!selectedImageId) return;
        const selected = images.find(img => img.id === selectedImageId);
        if (!selected) return;
        images = images.filter(img => img.id !== selectedImageId).concat([selected]);
        updateImageList();
        render();
    });
}

if (sendToBackBtn) {
    sendToBackBtn.addEventListener('click', () => {
        if (!selectedImageId) return;
        const selected = images.find(img => img.id === selectedImageId);
        if (!selected) return;
```

```
        images = [selected].concat(images.filter(img => img.id !== selectedImageId));
        updateImageList();
        render();
    });
}

if (composerRemoveBgBtn) {
    let isRemovingBg = false;
    composerRemoveBgBtn.addEventListener('click', async () => {
        const selected = images.find(img => img.id === selectedImageId);
        if (!selected || isRemovingBg) return;

        if (composerToleranceContainer && composerToleranceContainer.style.display === '
            composerToleranceContainer.style.display = 'block';
        }

        if (!window.DocProcessor || !window.DocProcessor.removeBackground) {
            alert('                    .  ';(
            return;
        }

        const origText = composerRemoveBgBtn.innerHTML;
        composerRemoveBgBtn.innerHTML = '      ...  ';
        composerRemoveBgBtn.disabled = true;
        isRemovingBg = true;

        try {
            const tol = parseInt(composerBgTol ? composerBgTol.value : 30) || 30;
            const resCanvas = await window.DocProcessor.removeBackground(selected.img, t
            const pngUrl = resCanvas.toDataURL('image/png');

            const newImg = new Image();
            newImg.onload = () => {
                selected.img = newImg;
                updateImageList();
                render();
            };
            newImg.src = pngUrl;
        } catch (err) {
            console.error('BG removal error:', err);
            alert('        :    ' + (err.message || err));
        } finally {
            isRemovingBg = false;
            composerRemoveBgBtn.innerHTML = origText;
            composerRemoveBgBtn.disabled = false;
        }
```

```
    });
}

if (composerBgTol) {
    composerBgTol.addEventListener('input', () => {
        if (composerBgTolVal) composerBgTolVal.textContent = composerBgTol.value;
    });
}

// --- Drag and Drop files onto canvas ---
['dragenter', 'dragover'].forEach(evt => {
    paperCanvasWrapper.addEventListener(evt, (e) => {
        e.preventDefault();
        e.stopPropagation();
        paperCanvasWrapper.classList.add('drag-over');
    });
});

['dragleave', 'drop'].forEach(evt => {
    paperCanvasWrapper.addEventListener(evt, (e) => {
        e.preventDefault();
        e.stopPropagation();
        paperCanvasWrapper.classList.remove('drag-over');
    });
});

paperCanvasWrapper.addEventListener('drop', (e) => {
    if (e.dataTransfer && e.dataTransfer.files) {
        const files = Array.from(e.dataTransfer.files).filter(f => f.type.startsWith('im
        files.forEach(f => addImage(f));
    }
});

// --- Check for imported images from main scanner ---
function checkImportedImages() {
    try {
        const raw = localStorage.getItem('composer_import_image');
        if (raw) {
            localStorage.removeItem('composer_import_image');
            const parsed = JSON.parse(raw);
            if (parsed && parsed.dataUrl) {
                const img = new Image();
                img.onload = () => {
                    const dims = getPaperDimensions();
                    const naturalW = img.naturalWidth || img.width;
                    const naturalH = img.naturalHeight || img.height;
```

```
                        const aspect = naturalW / naturalH;
                        let initW = (dims.w - MARGIN * 2) * 0.5;
                        let initH = initW / aspect;

                        const item = {
                            id: 'img_' + (nextImageId++),
                            img: img,
                            name: parsed.name || '         ',
                            x: MARGIN,
                            y: MARGIN,
                            width: Math.round(initW * 10) / 10,
                            height: Math.round(initH * 10) / 10,
                            naturalWidth: naturalW,
                            naturalHeight: naturalH
                        };
                        images.push(item);
                        selectedImageId = item.id;
                        updateImageList();
                        render();
                    };
                    img.src = parsed.dataUrl;
                }
            }
        } catch (e) {
            console.error('Import error:', e);
        }
    }

    // Initialize Layout
    updateBaseScale();
    resizeCanvases();
    render();
    checkImportedImages();
});
```

فایل index.html:

```
<!DOCTYPE html>
<html lang="fa" dir="rtl">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>         <title/>
    <link rel="stylesheet" href="styles.css">
```

```
    <!-- JSZip for ZIP download -->
    <script src="jszip.min.js"></script>
    <!-- jsPDF for PDF export -->
    <script src="jspdf.umd.min.js"></script>
</head>
<body>
    <!-- Removed background effects -->
    <main class="container">
        <header>
            <h1>          <h1/>
            <p>                    <p/>
        </header>

        <div class="upload-section glass-panel" id="drop-zone">
            <div class="upload-content">
                <svg viewBox="0 0 24 24" class="upload-icon" fill="none" stroke="currentColo
                    <path d="M21 15v4a2 2 0 0 1-2 2H5a2 2 0 0 1-2-2v-4"></path>
                    <polyline points="17 8 12 3 7 8"></polyline>
                    <line x1="12" y1="3" x2="12" y2="15"></line>
                </svg>
                <h3>               <h3/>
                <p>             <p/>
                <input type="file" id="file-input" multiple accept="image/*" hidden>
            </div>
        </div>

        <div class="workspace hidden" id="workspace">
            <aside class="controls glass-panel">
                <div class="control-group">
                    <h3>   <h3/>
                    <div class="mode-selector">
                        <label class="radio-label">
                            <input type="radio" name="mode" value="color" checked>
                            <span class="radio-button">      <span/>(    )
                        </label>
                        <label class="radio-label">
                            <input type="radio" name="mode" value="bw">
                            <span class="radio-button">        <span/>
                        </label>
                        <label class="radio-label">
                            <input type="radio" name="mode" value="enhance">
                            <span class="radio-button">    <span/>
                        </label>
                    </div>
                </div>
```

```
<div class="control-group">
    <h3>    </h3>

    <div class="slider-container">
        <div class="slider-header">
            <label for="brightness">    </label>
            <span id="brightness-val" class="slider-val">0</span>
        </div>
        <input type="range" id="brightness" min="-100" max="100" value="0">
    </div>

    <div class="slider-container">
        <div class="slider-header">
            <label for="contrast">    </label>
            <span id="contrast-val" class="slider-val">0</span>
        </div>
        <input type="range" id="contrast" min="-100" max="100" value="0">
    </div>

    <div class="slider-container" id="threshold-container" style="opacity: (
        <div class="slider-header">
            <label for="threshold">      )   <label/>(
            <span id="threshold-val" class="slider-val">128</span>
        </div>
        <input type="range" id="threshold" min="0" max="255" value="128">
    </div>

    <div class="slider-container">
        <div class="slider-header">
            <label for="shadow">   <label/>
            <span id="shadow-val" class="slider-val">70</span>
        </div>
        <input type="range" id="shadow" min="0" max="100" value="70">
    </div>
</div>

<div class="control-group">
    <h3>     <h3/>
    <div class="tools-container">
        <div style="display: flex; gap: 5px; margin-bottom: 0.5rem;">
            <button id="undo-btn" class="btn secondary-btn" disabled>   </bu
            <button id="redo-btn" class="btn secondary-btn" disabled>  </but
        </div>
        <button id="reset-image-btn" class="btn secondary-btn" style="margin

        <button id="crop-toggle" class="btn secondary-btn">        <button/>
```

```
            <button id="rotate-btn" class="btn secondary-btn" style="margin-top
            <button id="crop-apply" class="btn primary-btn" style="display:none

            <button id="eraser-toggle" class="btn secondary-btn" style="margin-t
            <button id="eraser-clear" class="btn secondary-btn" style="display:n

            <div class="slider-container" id="brush-size-container" style="displ
                <div class="slider-header">
                    <label for="brush-size">    <label/>
                    <span id="brush-size-val" class="slider-val">50</span>
                </div>
                <input type="range" id="brush-size" min="10" max="300" value="50
            </div>

            <div class="slider-container" id="brush-intensity-container" style="
                <div class="slider-header">
                    <label for="brush-intensity">   <label/>
                    <span id="brush-intensity-val" class="slider-val">50</span>
                </div>
                <input type="range" id="brush-intensity" min="1" max="100" value
            </div>

            <hr style="margin: 1rem 0; border-color: var(--panel-border);">
            <h4 style="margin-bottom: 0.5rem; font-size: 0.9rem;">   <h4/>
            <button id="bg-remove-btn" class="btn secondary-btn">   <button/>

            <div class="slider-container" id="bg-tolerance-container" style="mar
                <div class="slider-header">
                    <label for="bg-tolerance">    <label/>
                    <span id="bg-tolerance-val" class="slider-val">30</span>
                </div>
                <input type="range" id="bg-tolerance" min="5" max="80" value="30
            </div>

            <div class="slider-container" id="bg-smooth-container">
                <div class="slider-header">
                    <label for="bg-smooth">   <label/>
                    <span id="bg-smooth-val" class="slider-val">3</span>
                </div>
                <input type="range" id="bg-smooth" min="0" max="20" value="3">
            </div>
        </div>
    </div>

    <div class="actions">
        <div class="format-selector">
```

```
                <label>   <label/>:
                <select id="export-format">
                    <option value="image/png" selected>PNG <option/>(  )
                    <option value="image/jpeg">JPEG</option>
                </select>
            </div>

            <div class="control-group" style="margin-top: 0.5rem;">
                <h3>    PDF</h3>
                <div class="pdf-size-selector">
                    <label class="radio-label">
                        <input type="radio" name="pdf-size" value="id-card">
                        <span class="radio-button">   <span/>
                    </label>
                    <label class="radio-label">
                        <input type="radio" name="pdf-size" value="a5">
                        <span class="radio-button">A5</span>
                    </label>
                    <label class="radio-label">
                        <input type="radio" name="pdf-size" value="a4" checked>
                        <span class="radio-button">A4</span>
                    </label>
                    <label class="radio-label">
                        <input type="radio" name="pdf-size" value="a3">
                        <span class="radio-button">A3</span>
                    </label>
                </div>
                <button id="download-pdf-btn" class="btn primary-btn" style="margin-
                <hr style="margin: 0.8rem 0; border-color: var(--panel-border);">
                <a href="composer.html" target="_blank" id="open-composer-link" clas
            </div>

            <button id="process-btn" class="btn primary-btn">   </button>
            <button id="download-all-btn" class="btn secondary-btn">       (ZIP)</bu
        </div>
    </aside>

    <section class="preview-area glass-panel">
        <div class="image-list-container">
            <div class="image-list-header">
                <h3>       )      <h3/>(
                <button class="btn add-more-btn" id="add-more-btn">
                    <svg viewBox="0 0 24 24" width="16" height="16" stroke="current0

                </button>
            </div>
```

```
            <div class="image-thumbnails" id="thumbnail-list">
                <!-- Thumbnails injected via JS -->
            </div>
        </div>

        <div class="comparison-view">
            <div class="preview-box">
                <div style="display: flex; justify-content: space-between; align-ite
                    <h4>    <h4/>
                    <div class="zoom-controls" id="zoom-controls">
                        <button id="crop-toggle-2" class="btn secondary-btn" style="
                        <button id="zoom-out-btn" class="btn zoom-btn" title="  <bu
                        <span id="zoom-level" class="zoom-level">100%</span>
                        <button id="zoom-in-btn" class="btn zoom-btn" title="  <but
                        <button id="zoom-fit-btn" class="btn zoom-btn" title="    "
                    </div>
                </div>
                <div class="canvas-container" id="original-container" style="positio
                    <canvas id="canvas-original"></canvas>
                    <svg id="crop-overlay" class="crop-overlay" style="display:none;
                        <polygon id="crop-polygon" points="10,10 90,10 90,90 10,90"
                        <circle class="crop-handle" id="crop-tl" cx="10" cy="10" r="
                        <circle class="crop-handle" id="crop-tr" cx="90" cy="10" r="
                        <circle class="crop-handle" id="crop-br" cx="90" cy="90" r="
                        <circle class="crop-handle" id="crop-bl" cx="10" cy="90" r="
                    </svg>
                </div>
            </div>
            <div class="preview-box">
                <h4>    <h4/>
                <div class="canvas-container" id="result-container">
                    <div class="spinner" id="processing-spinner"></div>
                    <canvas id="canvas-result"></canvas>
                </div>
                <button id="download-single-btn" class="btn icon-btn">
                    <svg viewBox="0 0 24 24" width="18" height="18" stroke="current

                </button>
            </div>
        </div>
    </section>
</div>

<footer class="app-footer">
    <p>          <strong>Behzad Farhadi</strong></p>
    <p style="margin-top: 4px;">    . .  -               <p/>
```

```
            <a href="https://daramet.com/behzadfarhadi" target="_blank" rel="noopener norefe

            </a>
        </footer>
    </main>

    <script src="processor.js"></script>
    <script src="app.js"></script>
</body>
</html>
```

فایل jspdf.umd.min.js:

```
/** @license
 *
 * jsPDF - PDF Document creation from JavaScript
 * Version 2.5.1 Built on 2022-01-28T15:37:57.789Z
 *                      CommitID 00000000
 *
 * Copyright (c) 2010-2021 James Hall <james@parall.ax>, https://github.com/MrRio/jsPDF
 *               2015-2021 yWorks GmbH, http://www.yworks.com
 *               2015-2021 Lukas Holländer <lukas.hollaender@yworks.com>, https://github.com
 *               2016-2018 Aras Abbasi <aras.abbasi@gmail.com>
 *               2010 Aaron Spike, https://github.com/acspike
 *               2012 Willow Systems Corporation, https://github.com/willowsystems
 *               2012 Pablo Hess, https://github.com/pablohess
 *               2012 Florian Jenett, https://github.com/fjenett
 *               2013 Warren Weckesser, https://github.com/warrenweckesser
 *               2013 Youssef Beddad, https://github.com/lifof
 *               2013 Lee Driscoll, https://github.com/lsdriscoll
 *               2013 Stefan Slonevskiy, https://github.com/stefslon
 *               2013 Jeremy Morel, https://github.com/jmorel
 *               2013 Christoph Hartmann, https://github.com/chris-rock
 *               2014 Juan Pablo Gaviria, https://github.com/juanpgaviria
 *               2014 James Makes, https://github.com/dollaruw
 *               2014 Diego Casorran, https://github.com/diegocr
 *               2014 Steven Spungin, https://github.com/Flamenco
 *               2014 Kenneth Glassey, https://github.com/Gavvers
 *
 * Permission is hereby granted, free of charge, to any person obtaining
 * a copy of this software and associated documentation files (the
 * "Software"), to deal in the Software without restriction, including
 * without limitation the rights to use, copy, modify, merge, publish,
 * distribute, sublicense, and/or sell copies of the Software, and to
 * permit persons to whom the Software is furnished to do so, subject to
```

```
 * the following conditions:
 *
 * The above copyright notice and this permission notice shall be
 * included in all copies or substantial portions of the Software.
 *
 * THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND,
 * EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF
 * MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND
 * NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE
 * LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION
 * OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION
 * WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.
 *
 * Contributor(s):
 *    siefkenj, ahwolf, rickygu, Midnith, saintclair, eaparango,
 *    kim3er, mfo, alnorth, Flamenco
 */

!function(t,e){"object"==typeof exports&&"undefined"!=typeof module?e(exports):"function"==t
/**
   * A class to parse color values
   * @author Stoyan Stefanov <sstoo@gmail.com>
   * {@link   http://www.phpied.com/rgb-color-parser-in-javascript/}
   * @license Use it if you like it
   */function h(t){var e;t=t||"",this.ok=!1,"#"==t.charAt(0)&&(t=t.substr(1,6));t={aliceblue
/**
   * @license
   * Joseph Myers does not specify a particular license for his work.
   *
   * Author: Joseph Myers
   * Accessed from: http://www.myersdaily.org/joseph/javascript/md5.js
   *
   * Modified by: Owen Leong
   */
function f(t,e){var r=t[0],n=t[1],i=t[2],a=t[3];r=p(r,n,i,a,e[0],7,-680876936),a=p(a,r,n,i,e
/**
   * @license
   * FPDF is released under a permissive license: there is no usage restriction.
   * You may embed it freely in your application (commercial or not), with or
   * without modifications.
   *
   * Reference: http://www.fpdf.org/en/script/script37.php
   */function _(t,e){var r,n,i,a;if(t!==r){for(var o=(i=t,a=1+(256/t.length>>0),new Array(a+
/**
   * @license
   * Licensed under the MIT License.
```

```
   * http://opensource.org/licenses/mit-license
   * Author: Owen Leong (@owenl131)
   * Date: 15 Oct 2020
   * References:
   * https://www.cs.cmu.edu/~dst/Adobe/Gallery/anon21jul01-pdf-encryption.txt
   * https://github.com/foliojs/pdfkit/blob/master/lib/security.js
   * http://www.fpdf.org/en/script/script37.php
   */var P={print:4,modify:8,copy:16,"annot-forms":32};function k(t,e,r,n){this.v=1,this.r=2
/**
   * @license
   * Copyright (c) 2014 Steven Spungin (TwelveTone LLC)  steven@twelvetone.tv
   *
   * Licensed under the MIT License.
   * http://opensource.org/licenses/mit-license
   */
function(t){var e=function(t){if(void 0!==t&&""!=t)return!0};M.API.events.push(["addPage",fu
/**
   * @license
   * Copyright (c) 2017 Aras Abbasi
   *
   * Licensed under the MIT License.
   * http://opensource.org/licenses/mit-license
   */
function(t){var e={1569:[65152],1570:[65153,65154],1571:[65155,65156],1572:[65157,65158],157
/** @license
   * jsPDF Autoprint Plugin
   *
   * Licensed under the MIT License.
   * http://opensource.org/licenses/mit-license
   */
function(t){t.autoPrint=function(t){var e;switch((t=t||{}).variant=t.variant||"non-conform",
/**
   * @license
   * Copyright (c) 2014 Steven Spungin (TwelveTone LLC)  steven@twelvetone.tv
   *
   * Licensed under the MIT License.
   * http://opensource.org/licenses/mit-license
   */
function(t){var e=function(){var t=void 0;Object.defineProperty(this,"pdf",{get:function(){r
/**
   * @license
   * jsPDF filters PlugIn
   * Copyright (c) 2014 Aras Abbasi
   *
   * Licensed under the MIT License.
   * http://opensource.org/licenses/mit-license
```

```
   */!function(t){var e=function(t){var e,r,n,i,a,o,s,c,u,l;for(/[^\x00-\xFF]/.test(t),r=[]
/**
   * @license
   * jsPDF fileloading PlugIn
   * Copyright (c) 2018 Aras Abbasi (aras.abbasi@gmail.com)
   *
   * Licensed under the MIT License.
   * http://opensource.org/licenses/mit-license
   */
function(t){t.loadFile=function(t,e,r){return function(t,e,r){e=!1!==e,r="function"==typeof
/**
   * @license
   * ====================================================================
   * Copyright (c) 2013 Youssef Beddad, youssef.beddad@gmail.com
   *
   * Permission is hereby granted, free of charge, to any person obtaining
   * a copy of this software and associated documentation files (the
   * "Software"), to deal in the Software without restriction, including
   * without limitation the rights to use, copy, modify, merge, publish,
   * distribute, sublicense, and/or sell copies of the Software, and to
   * permit persons to whom the Software is furnished to do so, subject to
   * the following conditions:
   *
   * The above copyright notice and this permission notice shall be
   * included in all copies or substantial portions of the Software.
   *
   * THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND,
   * EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF
   * MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND
   * NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE
   * LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION
   * OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION
   * WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.
   * ====================================================================
   */
function(t){var e,r,n;t.addJS=function(t){return n=t,this.internal.events.subscribe("postPut
/**
   * @license
   * Copyright (c) 2014 Steven Spungin (TwelveTone LLC)  steven@twelvetone.tv
   *
   * Licensed under the MIT License.
   * http://opensource.org/licenses/mit-license
   */
function(t){var e;t.events.push(["postPutResources",function(){var t=this,r=/^(\d+) 0 obj$/
/**
   * @license
```

```
   *
   * Licensed under the MIT License.
   * http://opensource.org/licenses/mit-license
   */
function(t){var e=[192,193,194,195,196,197,198,199];t.processJPEG=function(t,r,n,i,a,o){var
/**
   * @license
   *
   * Copyright (c) 2014 James Robb, https://github.com/jamesbrobb
   *
   * Permission is hereby granted, free of charge, to any person obtaining
   * a copy of this software and associated documentation files (the
   * "Software"), to deal in the Software without restriction, including
   * without limitation the rights to use, copy, modify, merge, publish,
   * distribute, sublicense, and/or sell copies of the Software, and to
   * permit persons to whom the Software is furnished to do so, subject to
   * the following conditions:
   *
   * The above copyright notice and this permission notice shall be
   * included in all copies or substantial portions of the Software.
   *
   * THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND,
   * EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF
   * MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND
   * NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE
   * LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION
   * OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION
   * WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.
   * ==========================================================
   */
/**
   * @license
   * (c) Dean McNamee <dean@gmail.com>, 2013.
   *
   * https://github.com/deanm/omggif
   *
   * Permission is hereby granted, free of charge, to any person obtaining a copy
   * of this software and associated documentation files (the "Software"), to
   * deal in the Software without restriction, including without limitation the
   * rights to use, copy, modify, merge, publish, distribute, sublicense, and/or
   * sell copies of the Software, and to permit persons to whom the Software is
   * furnished to do so, subject to the following conditions:
   *
   * The above copyright notice and this permission notice shall be included in
   * all copies or substantial portions of the Software.
   *
```

```
   * THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
   * IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
   * FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
   * AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
   * LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING
   * FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS
   * IN THE SOFTWARE.
   *
   * omggif is a JavaScript implementation of a GIF 89a encoder and decoder,
   * including animation and compression.  It does not rely on any specific
   * underlying system, so should run in the browser, Node, or Plask.
   */
function Be(t){var e=0;if(71!==t[e++]||73!==t[e++]||70!==t[e++]||56!==t[e++]||56!=(t[e++]+1&
/**
   * @license
    Copyright (c) 2008, Adobe Systems Incorporated
    All rights reserved.

    Redistribution and use in source and binary forms, with or without
    modification, are permitted provided that the following conditions are
    met:

    * Redistributions of source code must retain the above copyright notice,
      this list of conditions and the following disclaimer.

    * Redistributions in binary form must reproduce the above copyright
      notice, this list of conditions and the following disclaimer in the
      documentation and/or other materials provided with the distribution.

    * Neither the name of Adobe Systems Incorporated nor the names of its
      contributors may be used to endorse or promote products derived from
      this software without specific prior written permission.

    THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS
    IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO,
    THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
    PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT OWNER OR
    CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL,
    EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO,
    PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR
    PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF
    LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING
    NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS
    SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
  */function Ee(t){var e,r,n,i,a,o=Math.floor,s=new Array(64),c=new Array(64),u=new Array(64
/**
```

```
   * @license
   * Copyright (c) 2017 Aras Abbasi
   *
   * Licensed under the MIT License.
   * http://opensource.org/licenses/mit-license
   */function qe(t,e){if(this.pos=0,this.buffer=t,this.datav=new DataView(t.buffer),this.is_
/** @license
     * Copyright (c) 2017 Dominik Homberger
    Permission is hereby granted, free of charge, to any person obtaining a copy of this sof
    The above copyright notice and this permission notice shall be included in all copies or
    THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCL
    https://webpjs.appspot.com
    WebPRiffParser dominikhlbg@gmail.com
    */
function(t,e,r,n){for(var i=0;i<n;i++)if(t[e+i]!=r.charCodeAt(i))return!0;return!1}(t,e,"RIF
/**
   * @license
   * Copyright (c) 2018 Aras Abbasi
   *
   * Licensed under the MIT License.
   * http://opensource.org/licenses/mit-license
   */
function(t){t.processBMP=function(e,r,n,i){var a=new qe(e,!1),o=a.width,s=a.height,c={data:a
/**
   * @license
   * Copyright (c) 2019 Aras Abbasi
   *
   * Licensed under the MIT License.
   * http://opensource.org/licenses/mit-license
   */
function(t){t.processWEBP=function(e,r,n,i){var a=new De(e,!1),o=a.width,s=a.height,c={data:
/**
   * @license
   *
   * Copyright (c) 2021 Antti Palola, https://github.com/Pantura
   *
   * Permission is hereby granted, free of charge, to any person obtaining
   * a copy of this software and associated documentation files (the
   * "Software"), to deal in the Software without restriction, including
   * without limitation the rights to use, copy, modify, merge, publish,
   * distribute, sublicense, and/or sell copies of the Software, and to
   * permit persons to whom the Software is furnished to do so, subject to
   * the following conditions:
   *
   * The above copyright notice and this permission notice shall be
   * included in all copies or substantial portions of the Software.
```

```
   *
   * THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND,
   * EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF
   * MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND
   * NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE
   * LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION
   * OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION
   * WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.
   * ======================================================================
   */
function(t){t.processRGBA=function(t,e,r){for(var n=t.data,i=n.length,a=new Uint8Array(i/4*3
/**
   * @license
   * Licensed under the MIT License.
   * http://opensource.org/licenses/mit-license
   */
function(t){t.setLanguage=function(t){return void 0===this.internal.languageSettings&&(this.
/**
   * @license
   * Licensed under the MIT License.
   * http://opensource.org/licenses/mit-license
   */
function(t){var e=function(t){for(var e=t.length,r=new Uint8Array(e),n=0;n<e;n++)r[n]=t.char
/**
   * @license
   * ======================================================================
   * Copyright (c) 2013 Eduardo Menezes de Morais, eduardo.morais@usp.br
   *
   * Permission is hereby granted, free of charge, to any person obtaining
   * a copy of this software and associated documentation files (the
   * "Software"), to deal in the Software without restriction, including
   * without limitation the rights to use, copy, modify, merge, publish,
   * distribute, sublicense, and/or sell copies of the Software, and to
   * permit persons to whom the Software is furnished to do so, subject to
   * the following conditions:
   *
   * The above copyright notice and this permission notice shall be
   * included in all copies or substantial portions of the Software.
   *
   * THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND,
   * EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF
   * MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND
   * NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE
   * LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION
   * OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION
   * WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.
```

```
   * ====================================================================
   */
function(t){t.putTotalPages=function(t){var e,r=0;parseInt(this.internal.getFont().id.substr
/** ====================================================================
   * @license
   * jsPDF XMP metadata plugin
   * Copyright (c) 2016 Jussi Utunen, u-jussi@suomi24.fi
   *
   * Permission is hereby granted, free of charge, to any person obtaining
   * a copy of this software and associated documentation files (the
   * "Software"), to deal in the Software without restriction, including
   * without limitation the rights to use, copy, modify, merge, publish,
   * distribute, sublicense, and/or sell copies of the Software, and to
   * permit persons to whom the Software is furnished to do so, subject to
   * the following conditions:
   *
   * The above copyright notice and this permission notice shall be
   * included in all copies or substantial portions of the Software.
   *
   * THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND,
   * EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF
   * MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND
   * NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE
   * LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION
   * OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION
   * WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.
   * ====================================================================
   */
function(t){var e=function(){var t='<rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syn
/**
   * @license
   * jsPDF virtual FileSystem functionality
   *
   * Licensed under the MIT License.
   * http://opensource.org/licenses/mit-license
   */
function(t){var e=function(){return void 0===this.internal.vFS&&(this.internal.vFS={}),!0};t
/**
   * @license
   * Unicode Bidi Engine based on the work of Alex Shensis (@asthensis)
   * MIT License
   */
function(t){t.__bidiEngine__=t.prototype.__bidiEngine__=function(t){var r,n,i,a,o,s,c,u=e,l=
//# sourceMappingURL=jspdf.umd.min.js.map
```

فایل: jszip.min.js

```
/*!

JSZip v3.10.1 - A JavaScript class for generating and reading zip files
<http://stuartk.com/jszip>

(c) 2009-2016 Stuart Knightley <stuart [at] stuartk.com>
Dual licenced under the MIT license or GPLv3. See https://raw.github.com/Stuk/jszip/main/LIC

JSZip uses the library pako released under the MIT license :
https://github.com/nodeca/pako/blob/main/LICENSE
*/

!function(e){if("object"==typeof exports&&"undefined"!=typeof module)module.exports=e();else
```

فایل: manifest.json

```
{
  "manifest_version": 3,
  "name": "Smart Document Scanner -         "    ,
  "version": "1.1.0",
  "description": "Offline document scanner: convert photos to clean scans, crop, erase, and
  "author": "Behzad Farhadi",
  "homepage_url": "https://daramet.com/behzadfarhadi",
  "action": {
    "default_icon": {
      "16": "icons/icon-16.png",
      "32": "icons/icon-32.png",
      "48": "icons/icon-48.png",
      "128": "icons/icon-128.png"
    },
    "default_title":    "     "
  {,
  "background": {
    "service_worker": "background.js"
  },
  "icons": {
    "16": "icons/icon-16.png",
    "32": "icons/icon-32.png",
    "48": "icons/icon-48.png",
    "128": "icons/icon-128.png"
  },
  "permissions": [],
  "offline_enabled": true,
  "content_security_policy": {
```

```
    "extension_pages": "script-src 'self'; object-src 'self'"
  }
}
```

فایل processor.js:

```
// Image Processing Algorithms (Canvas API)
// Optimized for phone-captured document photos

class ImageProcessor {
    constructor() {
        this.canvas = document.createElement('canvas');
        this.ctx = this.canvas.getContext('2d', { willReadFrequently: true });
    }

    async process(imageElement, options) {
        return new Promise((resolve, reject) => {
            setTimeout(() => {
                try {
                    const width = imageElement.naturalWidth || imageElement.width;
                    const height = imageElement.naturalHeight || imageElement.height;

                    this.canvas.width = width;
                    this.canvas.height = height;

                    this.ctx.drawImage(imageElement, 0, 0, width, height);
                    let processedData = this.ctx.getImageData(0, 0, width, height);

                    const brightness = parseInt(options.brightness) || 0;
                    const contrast = parseInt(options.contrast) || 0;
                    const threshold = parseInt(options.threshold) || 128;
                    const shadowStrength = (parseInt(options.shadow) || 0) / 100;

                    if (options.mode === 'bw') {
                        processedData = this.processBW(processedData, width, height, shadowS
                    } else if (options.mode === 'color') {
                        processedData = this.processColor(processedData, width, height, shad
                    } else if (options.mode === 'enhance') {
                        processedData = this.processEnhance(processedData, width, height, sh
                    }

                    if (brightness !== 0 || contrast !== 0) {
                        processedData = this.applyBrightnessContrast(processedData, brightne
                    }
```

```
                if (options.eraserMask) {
                    processedData = this.applySmartEraser(processedData, options.eraserM
                }

                this.ctx.putImageData(processedData, 0, 0);
                resolve(this.canvas);
            } catch (err) {
                reject(err);
            }
        }, 50);
    });
}

// ============================================================
// B&W Scan - Clean black text on white background
// ============================================================
processBW(imgData, w, h, shadowStrength, thresholdValue) {
    const data = imgData.data;
    let gray = new Float32Array(w * h);

    // Convert to grayscale (float for precision)
    for (let i = 0, j = 0; i < data.length; i += 4, j++) {
        gray[j] = data[i] * 0.299 + data[i + 1] * 0.587 + data[i + 2] * 0.114;
    }

    // Shadow removal with large-kernel background estimation
    if (shadowStrength > 0) {
        gray = this.removeShadows(gray, w, h, shadowStrength);
    }

    // Adaptive threshold using Sauvola-inspired method
    // Use large window for document-scale features
    const windowSize = Math.max(15, Math.floor(Math.min(w, h) / 15));
    // Map slider: 0=very sensitive (more black), 255=less sensitive (more white)
    // k controls how much darker a pixel must be vs local mean to be "ink"
    const k = 0.1 + (thresholdValue / 255) * 0.5; // range 0.1 to 0.6

    const localMean = this.multiPassBoxBlur(gray, w, h, windowSize);
    const localStdDev = this.computeLocalStdDev(gray, localMean, w, h, windowSize);

    const newData = new ImageData(w, h);
    const out = newData.data;

    for (let i = 0, j = 0; i < out.length; i += 4, j++) {
        // Sauvola: threshold = mean * (1 + k * (stddev / R - 1))
        const R = 128;
```

```
            const localThresh = localMean[j] * (1.0 + k * (localStdDev[j] / R - 1.0));
            const val = gray[j] < localThresh ? 0 : 255;
            out[i] = val;
            out[i + 1] = val;
            out[i + 2] = val;
            out[i + 3] = data[i + 3];
        }

        return newData;
    }

    // ============================================================
    // Color Scan - Clean document with preserved grayscale tones
    // This is what CamScanner-like apps typically produce
    // ============================================================
    processColor(imgData, w, h, shadowStrength) {
        const data = imgData.data;
        let gray = new Float32Array(w * h);

        for (let i = 0, j = 0; i < data.length; i += 4, j++) {
            gray[j] = data[i] * 0.299 + data[i + 1] * 0.587 + data[i + 2] * 0.114;
        }

        // Strong shadow removal
        if (shadowStrength > 0) {
            gray = this.removeShadows(gray, w, h, shadowStrength);
        }

        // Contrast stretching based on histogram
        gray = this.histogramStretch(gray, w, h);

        // Gamma correction to brighten midtones (make paper whiter)
        const gamma = 1.4;
        for (let i = 0; i < gray.length; i++) {
            gray[i] = 255 * Math.pow(gray[i] / 255, 1 / gamma);
        }

        const newData = new ImageData(w, h);
        const out = newData.data;

        for (let i = 0, j = 0; i < out.length; i += 4, j++) {
            const val = this.clamp(gray[j]);
            out[i] = val;
            out[i + 1] = val;
            out[i + 2] = val;
            out[i + 3] = data[i + 3];
```

```
    }

    return newData;
}

// ============================================================
// Enhance - Light cleanup, keeps original character
// ============================================================
processEnhance(imgData, w, h, shadowStrength) {
    const data = imgData.data;

    // Per-channel shadow removal for color preservation
    let r = new Float32Array(w * h);
    let g = new Float32Array(w * h);
    let b = new Float32Array(w * h);

    for (let i = 0, j = 0; i < data.length; i += 4, j++) {
        r[j] = data[i];
        g[j] = data[i + 1];
        b[j] = data[i + 2];
    }

    if (shadowStrength > 0) {
        const mildStrength = shadowStrength * 0.6;
        r = this.removeShadows(r, w, h, mildStrength);
        g = this.removeShadows(g, w, h, mildStrength);
        b = this.removeShadows(b, w, h, mildStrength);
    }

    // Light contrast stretch per channel
    r = this.histogramStretch(r, w, h, 2, 98);
    g = this.histogramStretch(g, w, h, 2, 98);
    b = this.histogramStretch(b, w, h, 2, 98);

    const newData = new ImageData(w, h);
    const out = newData.data;

    for (let i = 0, j = 0; i < out.length; i += 4, j++) {
        out[i] = this.clamp(r[j]);
        out[i + 1] = this.clamp(g[j]);
        out[i + 2] = this.clamp(b[j]);
        out[i + 3] = data[i + 3];
    }

    return newData;
}
```

```
// ============================================================
// CORE: Shadow Removal via Background Division
// ============================================================
removeShadows(channel, w, h, strength) {
    // Use very large blur to estimate background illumination
    // Larger radius = better shadow estimation for big shadows
    const radius = Math.max(20, Math.floor(Math.min(w, h) / 5));
    const background = this.multiPassBoxBlur(channel, w, h, radius);

    const result = new Float32Array(w * h);

    // Find max background value for normalization
    let maxBg = 0;
    for (let i = 0; i < background.length; i++) {
        if (background[i] > maxBg) maxBg = background[i];
    }
    if (maxBg === 0) maxBg = 1;

    for (let i = 0; i < channel.length; i++) {
        const bg = background[i] || 1;
        // Divide by background and normalize
        const corrected = (channel[i] / bg) * maxBg;
        // Blend based on strength
        result[i] = channel[i] * (1 - strength) + corrected * strength;
    }

    return result;
}

// ============================================================
// Histogram-based Contrast Stretching
// ============================================================
histogramStretch(channel, w, h, lowPercentile = 1, highPercentile = 99) {
    const size = w * h;

    // Build histogram
    const histogram = new Int32Array(256);
    for (let i = 0; i < size; i++) {
        const val = Math.min(255, Math.max(0, Math.round(channel[i])));
        histogram[val]++;
    }

    // Find percentile bounds
    const lowCount = Math.floor(size * lowPercentile / 100);
    const highCount = Math.floor(size * highPercentile / 100);
```

```
    let cumulative = 0;
    let lowVal = 0;
    for (let i = 0; i < 256; i++) {
        cumulative += histogram[i];
        if (cumulative >= lowCount) { lowVal = i; break; }
    }

    cumulative = 0;
    let highVal = 255;
    for (let i = 255; i >= 0; i--) {
        cumulative += histogram[i];
        if (cumulative >= (size - highCount)) { highVal = i; break; }
    }

    if (highVal <= lowVal) highVal = lowVal + 1;

    // Stretch
    const result = new Float32Array(size);
    const range = highVal - lowVal;
    for (let i = 0; i < size; i++) {
        result[i] = ((channel[i] - lowVal) / range) * 255;
        if (result[i] < 0) result[i] = 0;
        if (result[i] > 255) result[i] = 255;
    }

    return result;
}

// ============================================================
// Multi-pass Box Blur (approximates Gaussian)
// 3 passes of box blur ≈ Gaussian blur
// ============================================================
multiPassBoxBlur(source, w, h, radius) {
    let current = new Float32Array(source);
    // 3 passes for better Gaussian approximation
    const passes = 3;
    const passRadius = Math.max(1, Math.floor(radius / passes));

    for (let p = 0; p < passes; p++) {
        current = this.boxBlurFloat(current, w, h, passRadius);
    }
    return current;
}

// Single-pass box blur on Float32Array
```

```
boxBlurFloat(source, w, h, r) {
    const temp = new Float32Array(w * h);
    const target = new Float32Array(w * h);

    // Horizontal pass
    for (let y = 0; y < h; y++) {
        const rowOffset = y * w;
        let sum = 0;
        const kernelSize = 2 * r + 1;

        // Initialize sum for first pixel
        for (let i = -r; i <= r; i++) {
            sum += source[rowOffset + Math.min(w - 1, Math.max(0, i))];
        }

        for (let x = 0; x < w; x++) {
            temp[rowOffset + x] = sum / kernelSize;
            const addIdx = Math.min(w - 1, x + r + 1);
            const remIdx = Math.max(0, x - r);
            sum += source[rowOffset + addIdx] - source[rowOffset + remIdx];
        }
    }

    // Vertical pass
    for (let x = 0; x < w; x++) {
        let sum = 0;
        const kernelSize = 2 * r + 1;

        for (let i = -r; i <= r; i++) {
            sum += temp[Math.min(h - 1, Math.max(0, i)) * w + x];
        }

        for (let y = 0; y < h; y++) {
            target[y * w + x] = sum / kernelSize;
            const addIdx = Math.min(h - 1, y + r + 1);
            const remIdx = Math.max(0, y - r);
            sum += temp[addIdx * w + x] - temp[remIdx * w + x];
        }
    }

    return target;
}

// ============================================================
// Compute Local Standard Deviation for Sauvola threshold
// ============================================================
```

```
computeLocalStdDev(gray, localMean, w, h, radius) {
    // Compute local mean of squared values
    const squared = new Float32Array(w * h);
    for (let i = 0; i < gray.length; i++) {
        squared[i] = gray[i] * gray[i];
    }
    const localMeanSq = this.multiPassBoxBlur(squared, w, h, radius);

    // stddev = sqrt(E[X^2] - E[X]^2)
    const stddev = new Float32Array(w * h);
    for (let i = 0; i < gray.length; i++) {
        const variance = Math.max(0, localMeanSq[i] - localMean[i] * localMean[i]);
        stddev[i] = Math.sqrt(variance);
    }
    return stddev;
}

// ============================================================
// Brightness & Contrast adjustment
// ============================================================
applyBrightnessContrast(imgData, brightness, contrast) {
    const data = imgData.data;
    const factor = (259 * (contrast + 255)) / (255 * (259 - contrast));

    for (let i = 0; i < data.length; i += 4) {
        data[i] = this.clamp(factor * (data[i] - 128) + 128 + brightness);
        data[i + 1] = this.clamp(factor * (data[i + 1] - 128) + 128 + brightness);
        data[i + 2] = this.clamp(factor * (data[i + 2] - 128) + 128 + brightness);
    }
    return imgData;
}

clamp(val) {
    return Math.min(255, Math.max(0, Math.round(val)));
}

// ============================================================
// Perspective Crop Math & Warp
// ============================================================
async perspectiveWarp(imageElement, pts, dstW, dstH) {
    // pts is array of 4 points: [{x,y}, {x,y}, {x,y}, {x,y}] (TL, TR, BR, BL)
    // Destination points are corners of dstW x dstH rectangle
    const dstPts = [
        {x: 0, y: 0},
        {x: dstW, y: 0},
        {x: dstW, y: dstH},
```

```
            {x: 0, y: dstH}
        ];

        // We want a matrix M that maps dstPts to pts (so we can inverse map)
        const M = this.getHomography(dstPts, pts);

        // Create canvas to read original image
        const tempCanvas = document.createElement('canvas');
        tempCanvas.width = imageElement.naturalWidth || imageElement.width;
        tempCanvas.height = imageElement.naturalHeight || imageElement.height;
        const tCtx = tempCanvas.getContext('2d', { willReadFrequently: true });
        tCtx.drawImage(imageElement, 0, 0);
        const srcData = tCtx.getImageData(0, 0, tempCanvas.width, tempCanvas.height);

        // Create destination image data
        const dstData = new ImageData(dstW, dstH);

        // Warp pixels
        this.warpPixels(srcData, dstData, M);

        // Return a new canvas with the warped image
        const resultCanvas = document.createElement('canvas');
        resultCanvas.width = dstW;
        resultCanvas.height = dstH;
        resultCanvas.getContext('2d').putImageData(dstData, 0, 0);
        return resultCanvas;
    }

    getHomography(src, dst) {
        // src and dst are array of 4 {x,y} points
        // Returns 3x3 matrix mapping src to dst
        // We solve A * h = 0
        const A = [];
        for (let i = 0; i < 4; i++) {
            const x = src[i].x;
            const y = src[i].y;
            const u = dst[i].x;
            const v = dst[i].y;
            A.push([-x, -y, -1, 0, 0, 0, x*u, y*u, u]);
            A.push([0, 0, 0, -x, -y, -1, x*v, y*v, v]);
        }

        // Solve A*h = 0 using Gaussian elimination on A^T * A or by simple elimination sinc
        // A is 8x9. We can set h_8 = 1 and solve for 8 variables.
        const B = [];
        for (let i = 0; i < 8; i++) {
```

```
            B.push([...A[i].slice(0,8), -A[i][8]]); // Move last column to RHS
        }

        const h = this.solveLinearSystem(B);
        h.push(1.0); // h_8

        return [
            [h[0], h[1], h[2]],
            [h[3], h[4], h[5]],
            [h[6], h[7], h[8]]
        ];
    }

    solveLinearSystem(M) {
        // M is 8x9 matrix (8 equations, 8 unknowns + 1 RHS)
        const n = 8;
        for (let i = 0; i < n; i++) {
            // Find pivot
            let maxRow = i;
            for (let k = i + 1; k < n; k++) {
                if (Math.abs(M[k][i]) > Math.abs(M[maxRow][i])) {
                    maxRow = k;
                }
            }

            // Swap
            const temp = M[i];
            M[i] = M[maxRow];
            M[maxRow] = temp;

            // Make diagonal 1
            const pivot = M[i][i];
            if (Math.abs(pivot) < 1e-10) return Array(n).fill(0); // Singular
            for (let j = i; j <= n; j++) {
                M[i][j] /= pivot;
            }

            // Eliminate other rows
            for (let k = 0; k < n; k++) {
                if (k === i) continue;
                const factor = M[k][i];
                for (let j = i; j <= n; j++) {
                    M[k][j] -= factor * M[i][j];
                }
            }
        }
```

```
    // Return RHS
    const result = [];
    for (let i = 0; i < n; i++) {
        result.push(M[i][n]);
    }
    return result;
}

warpPixels(src, dst, M) {
    const sw = src.width;
    const sh = src.height;
    const dw = dst.width;
    const dh = dst.height;

    const srcData = src.data;
    const dstData = dst.data;

    for (let y = 0; y < dh; y++) {
        for (let x = 0; x < dw; x++) {
            // Map dst(x,y) to src(u,v)
            const z = M[2][0] * x + M[2][1] * y + M[2][2];
            let u = (M[0][0] * x + M[0][1] * y + M[0][2]) / z;
            let v = (M[1][0] * x + M[1][1] * y + M[1][2]) / z;

            // Bilinear interpolation
            const u0 = Math.floor(u);
            const v0 = Math.floor(v);
            const u1 = u0 + 1;
            const v1 = v0 + 1;

            const fu = u - u0;
            const fv = v - v0;

            const dstIdx = (y * dw + x) * 4;

            if (u0 >= 0 && u1 < sw && v0 >= 0 && v1 < sh) {
                const i00 = (v0 * sw + u0) * 4;
                const i10 = (v0 * sw + u1) * 4;
                const i01 = (v1 * sw + u0) * 4;
                const i11 = (v1 * sw + u1) * 4;

                for (let c = 0; c < 4; c++) {
                    const val = (
                        srcData[i00 + c] * (1 - fu) * (1 - fv) +
                        srcData[i10 + c] * fu * (1 - fv) +
```

```
                        srcData[i01 + c] * (1 - fu) * fv +
                        srcData[i11 + c] * fu * fv
                    );
                    dstData[dstIdx + c] = val;
                }
            } else {
                // Out of bounds
                dstData[dstIdx] = 0;
                dstData[dstIdx+1] = 0;
                dstData[dstIdx+2] = 0;
                dstData[dstIdx+3] = 0;
            }
        }
    }
}

// ============================================================
// Smart Eraser
// ============================================================
applySmartEraser(imgData, maskCanvas, w, h) {
    const maskCtx = maskCanvas.getContext('2d', { willReadFrequently: true });
    const maskData = maskCtx.getImageData(0, 0, w, h).data;
    const data = imgData.data;

    for (let i = 0, j = 0; i < data.length; i += 4, j++) {
        const maskVal = maskData[i+3]; // Read Alpha channel for intensity
        if (maskVal > 0) {
            const r = data[i], g = data[i+1], b = data[i+2];
            const luma = r * 0.299 + g * 0.587 + b * 0.114;

            // Smart protection: if it's very dark (text), protection is 1.
            // If it's lighter (shadow/smudge), protection is 0.
            const protection = Math.max(0, Math.min(1, (120 - luma) / 60));

            const eraseFactor = (maskVal / 255) * (1 - protection);

            data[i] = r + (255 - r) * eraseFactor;
            data[i+1] = g + (255 - g) * eraseFactor;
            data[i+2] = b + (255 - b) * eraseFactor;
        }
    }
    return imgData;
}

// ============================================================
// Background Removal -
```

```
// ============================================================
async removeBackground(imageElement, tolerance = 30, edgeSmooth = 3) {
    return new Promise((resolve, reject) => {
        setTimeout(() => {
            try {
                const width = imageElement.naturalWidth || imageElement.width;
                const height = imageElement.naturalHeight || imageElement.height;

                const canvas = document.createElement('canvas');
                canvas.width = width;
                canvas.height = height;
                const ctx = canvas.getContext('2d', { willReadFrequently: true });
                ctx.drawImage(imageElement, 0, 0, width, height);

                const imgData = ctx.getImageData(0, 0, width, height);
                const data = imgData.data;

                //  .                          )      ( ×
                const blockSize = Math.min(20, Math.floor(width / 4), Math.floor(height
                const sampleCorner = (startX, startY) => {
                    let rSum = 0, gSum = 0, bSum = 0, count = 0;
                    const endY = Math.min(height, startY + blockSize);
                    const endX = Math.min(width, startX + blockSize);
                    for (let y = startY; y < endY; y++) {
                        for (let x = startX; x < endX; x++) {
                            const idx = (y * width + x) * 4;
                            rSum += data[idx];
                            gSum += data[idx + 1];
                            bSum += data[idx + 2];
                            count++;
                        }
                    }
                    return count > 0 ? { r: rSum / count, g: gSum / count, b: bSum / cou
                };

                const cTL = sampleCorner(0, 0);
                const cTR = sampleCorner(width - blockSize, 0);
                const cBL = sampleCorner(0, height - blockSize);
                const cBR = sampleCorner(width - blockSize, height - blockSize);

                //  .                       (Sobel edge detection)
                const gray = new Uint8Array(width * height);
                for (let i = 0, j = 0; i < data.length; i += 4, j++) {
                    gray[j] = (data[i] * 299 + data[i + 1] * 587 + data[i + 2] * 114) /
                }
```

```
const edgeMap = new Uint8Array(width * height);
for (let y = 1; y < height - 1; y++) {
    const rowOffset = y * width;
    for (let x = 1; x < width - 1; x++) {
        const idx = rowOffset + x;
        const p00 = gray[idx - width - 1], p01 = gray[idx - width], p02
        const p10 = gray[idx - 1],                                  p12
        const p20 = gray[idx + width - 1], p21 = gray[idx + width], p22

        const gx = (-p00 + p02) + 2 * (-p10 + p12) + (-p20 + p22);
        const gy = (-p00 - 2 * p01 - p02) + (p20 + 2 * p21 + p22);

        const mag = Math.min(255, Math.round(Math.sqrt(gx * gx + gy * gy
        edgeMap[idx] = mag;
    }
}

//
const edgeThreshold = 35;

//                tolerance ( -  )
const maxColorDist = tolerance * 2.8;
const colorDist = (r, g, b, corner) => {
    const dr = r - corner.r;
    const dg = g - corner.g;
    const db = b - corner.b;
    return Math.sqrt(dr * dr + dg * dg + db * db);
};

//    .       Flood-fill
const visited = new Uint8Array(width * height);
const isBackground = new Uint8Array(width * height);
const queue = new Int32Array(width * height);
let head = 0;
let tail = 0;

const addSeed = (x, y) => {
    if (x < 0 || x >= width || y < 0 || y >= height) return;
    const idx = y * width + x;
    if (visited[idx] === 0) {
        visited[idx] = 1;
        isBackground[idx] = 1;
        queue[tail++] = idx;
    }
};
```

```
//
addSeed(0, 0);
addSeed(width - 1, 0);
addSeed(0, height - 1);
addSeed(width - 1, height - 1);

const dx = [1, -1, 0, 0];
const dy = [0, 0, 1, -1];

while (head < tail) {
    const curIdx = queue[head++];
    const cx = curIdx % width;
    const cy = (curIdx / width) | 0;

    for (let d = 0; d < 4; d++) {
        const nx = cx + dx[d];
        const ny = cy + dy[d];

        if (nx < 0 || nx >= width || ny < 0 || ny >= height) continue;

        const nIdx = ny * width + nx;
        if (visited[nIdx]) continue;
        visited[nIdx] = 1;

        // .
        if (edgeMap[nIdx] > edgeThreshold) {
            continue;
        }

        //
        const pIdx = nIdx * 4;
        const r = data[pIdx];
        const g = data[pIdx + 1];
        const b = data[pIdx + 2];

        const minDist = Math.min(
            colorDist(r, g, b, cTL),
            colorDist(r, g, b, cTR),
            colorDist(r, g, b, cBL),
            colorDist(r, g, b, cBR)
        );

        if (minDist <= maxColorDist) {
            isBackground[nIdx] = 1;
            queue[tail++] = nIdx;
        }
```

```
                    }
                }

                //       :        =        =
                let alphaMask = new Uint8Array(width * height);
                for (let i = 0; i < width * height; i++) {
                    alphaMask[i] = isBackground[i] ? 0 : 255;
                }

                // .     Gaussian Blur
                const smoothRadius = Math.min(20, Math.max(0, Math.round(edgeSmooth)));
                if (smoothRadius > 0) {
                    alphaMask = this.applyAlphaMaskBlur(alphaMask, width, height, smooth
                }

                // .
                for (let i = 0; i < width * height; i++) {
                    const a = alphaMask[i];
                    const idx = i * 4;
                    data[idx + 3] = a < 3 ? 0 : Math.min(data[idx + 3], a);
                }

                // .
                ctx.putImageData(imgData, 0, 0);
                resolve(canvas);
            } catch (err) {
                reject(err);
            }
        }, 50);
    });
}

applyAlphaMaskBlur(mask, w, h, radius) {
    const sigma = Math.max(0.8, radius / 2.5);
    const kernelSize = 2 * radius + 1;
    const kernel = new Float32Array(kernelSize);
    let kSum = 0;
    for (let i = -radius; i <= radius; i++) {
        const val = Math.exp(-(i * i) / (2 * sigma * sigma));
        kernel[i + radius] = val;
        kSum += val;
    }
    for (let i = 0; i < kernelSize; i++) {
        kernel[i] /= kSum;
    }
```

```
        const temp = new Float32Array(w * h);
        const result = new Uint8Array(w * h);

        //
        for (let y = 0; y < h; y++) {
            const rowOffset = y * w;
            for (let x = 0; x < w; x++) {
                let sum = 0;
                for (let k = -radius; k <= radius; k++) {
                    const px = Math.min(w - 1, Math.max(0, x + k));
                    sum += mask[rowOffset + px] * kernel[k + radius];
                }
                temp[rowOffset + x] = sum;
            }
        }

        //
        for (let x = 0; x < w; x++) {
            for (let y = 0; y < h; y++) {
                let sum = 0;
                for (let k = -radius; k <= radius; k++) {
                    const py = Math.min(h - 1, Math.max(0, y + k));
                    sum += temp[py * w + x] * kernel[k + radius];
                }
                result[y * w + x] = Math.round(sum);
            }
        }

        return result;
    }
}

window.DocProcessor = new ImageProcessor();
```

:لیاف project_report.txt

هبلیدل گرزب،ندوب یاوتحم نیا لیاف دراو شرازگ .دشن

:لیاف styles.css

```
/* Local Vazirmatn Font */
@font-face {
    font-family: 'Vazirmatn';
    src: url('fonts/Vazirmatn-Regular.woff2') format('woff2');
    font-weight: 400;
```

```
    font-style: normal;
    font-display: swap;
}
@font-face {
    font-family: 'Vazirmatn';
    src: url('fonts/Vazirmatn-Bold.woff2') format('woff2');
    font-weight: 700;
    font-style: normal;
    font-display: swap;
}

:root {
    --bg-color: #f0f0f0;
    --panel-bg: #ffffff;
    --panel-border: #cccccc;
    --text-primary: #000000;
    --text-secondary: #333333;
    --accent: #0078d7;
    --accent-hover: #005a9e;
    --danger: #d13438;

    --btn-bg: #e1e1e1;
    --btn-border: #adadad;
    --btn-hover-bg: #e5f1fb;
    --btn-hover-border: #0078d7;

    --input-border: #cccccc;
    --input-focus: #0078d7;
}

* {
    box-sizing: border-box;
    margin: 0;
    padding: 0;
}

body {
    font-family: Tahoma, 'Segoe UI', 'Vazirmatn', sans-serif;
    background-color: var(--bg-color);
    color: var(--text-primary);
    min-height: 100vh;
    font-size: 14px;
}

.container {
    max-width: 1400px;
```

```
    margin: 0 auto;
    padding: 10px;
}

header {
    text-align: right;
    margin-bottom: 10px;
    padding-bottom: 10px;
    border-bottom: 1px solid var(--panel-border);
}

header h1 {
    font-size: 1.5rem;
    font-weight: normal;
    color: var(--text-primary);
    margin-bottom: 5px;
}

header p {
    color: var(--text-secondary);
    font-size: 0.9rem;
}

/* Upload Section */
.upload-section {
    padding: 20px;
    margin: 20px 0;
    text-align: center;
    background: var(--panel-bg);
    border: 1px dashed var(--panel-border);
    cursor: pointer;
}

.upload-section:hover {
    background: #f9f9f9;
}

.upload-icon {
    width: 48px;
    height: 48px;
    stroke: var(--text-secondary);
    margin-bottom: 10px;
}

/* Panels */
.glass-panel {
```

```
    background: var(--panel-bg);
    border: 1px solid var(--panel-border);
    padding: 15px;
}

/* Workspace Layout */
.workspace {
    display: flex;
    gap: 15px;
    align-items: flex-start;
}

.controls {
    flex: 0 0 300px;
    display: flex;
    flex-direction: column;
    gap: 15px;
}

.preview-area {
    flex: 1;
    display: flex;
    flex-direction: column;
    gap: 15px;
    min-width: 0;
}

/* Control Groups */
.control-group {
    border: 1px solid var(--panel-border);
    padding: 10px;
    background: #f9f9f9;
}

.control-group h3 {
    font-size: 1rem;
    font-weight: bold;
    margin-top: -20px;
    background: #f9f9f9;
    display: inline-block;
    padding: 0 5px;
    color: var(--text-secondary);
}

/* Form Controls */
.mode-selector, .pdf-size-selector {
```

```
    display: flex;
    flex-direction: column;
    gap: 8px;
    margin-top: 10px;
}

.radio-label {
    display: flex;
    align-items: center;
    gap: 8px;
    cursor: pointer;
}

.radio-label input[type="radio"] {
    margin: 0;
}

/* Sliders */
.slider-container {
    margin-bottom: 15px;
    margin-top: 10px;
}

.slider-header {
    display: flex;
    justify-content: space-between;
    margin-bottom: 5px;
    font-size: 0.9rem;
}

input[type="range"] {
    width: 100%;
}

/* Buttons */
.btn {
    width: 100%;
    padding: 6px 12px;
    font-family: inherit;
    font-size: 0.9rem;
    border: 1px solid var(--btn-border);
    background: var(--btn-bg);
    color: var(--text-primary);
    cursor: pointer;
    text-align: center;
    display: inline-flex;
```

```
    align-items: center;
    justify-content: center;
    gap: 5px;
}

.btn:hover {
    background: var(--btn-hover-bg);
    border-color: var(--btn-hover-border);
}

.primary-btn {
    background: var(--accent);
    color: white;
    border-color: var(--accent);
}

.primary-btn:hover {
    background: var(--accent-hover);
    border-color: var(--accent-hover);
}

.secondary-btn {
    /* Standard button style */
}

/* Select */
select {
    width: 100%;
    padding: 4px;
    border: 1px solid var(--input-border);
    font-family: inherit;
    margin-top: 5px;
}

/* Thumbnail List */
.image-list-container {
    border-bottom: 1px solid var(--panel-border);
    padding-bottom: 15px;
}

.image-list-header {
    display: flex;
    justify-content: space-between;
    align-items: center;
    margin-bottom: 10px;
}
```

```
.image-list-header h3 {
    font-size: 1rem;
}

.add-more-btn {
    width: auto;
    padding: 4px 8px;
}

.image-thumbnails {
    display: flex;
    gap: 10px;
    overflow-x: auto;
    padding-bottom: 5px;
}

.thumb-wrapper {
    position: relative;
    width: 60px;
    height: 80px;
    flex-shrink: 0;
    border: 1px solid var(--panel-border);
    cursor: pointer;
    background: #fff;
    padding: 2px;
}

.thumb-wrapper.active {
    border: 2px solid var(--accent);
    padding: 1px;
}

.thumb-wrapper img {
    width: 100%;
    height: 100%;
    object-fit: cover;
}

.remove-thumb {
    position: absolute;
    top: -5px;
    right: -5px;
    width: 16px;
    height: 16px;
    background: var(--danger);
```

```
    color: white;
    border: none;
    font-size: 12px;
    line-height: 16px;
    text-align: center;
    cursor: pointer;
}

/* Canvas Area */
.comparison-view {
    display: flex;
    gap: 15px;
    flex: 1;
}

.preview-box {
    flex: 1;
    display: flex;
    flex-direction: column;
    border: 1px solid var(--panel-border);
    background: #e6e6e6; /* Checkerboard alternative */
    padding: 5px;
}

.preview-box h4 {
    font-size: 0.9rem;
    padding: 5px;
    background: var(--panel-bg);
    border: 1px solid var(--panel-border);
    margin-bottom: 5px;
    text-align: center;
}

.canvas-container {
    flex: 1;
    background: url('
    position: relative;
    display: flex;
    align-items: center;
    justify-content: center;
    border: 1px solid #999;
    overflow: hidden;
    min-height: 400px;
}

canvas {
```

```
    max-width: 100%;
    max-height: 100%;
    box-shadow: 2px 2px 5px rgba(0,0,0,0.2);
    transform-origin: center center;
    transition: transform 0.15s ease;
}

.zoom-controls {
    display: flex;
    align-items: center;
    gap: 3px;
}

.zoom-btn {
    width: 26px;
    height: 26px;
    padding: 0;
    font-size: 16px;
    font-weight: bold;
    line-height: 1;
    display: flex;
    align-items: center;
    justify-content: center;
    min-width: unset;
}

.zoom-level {
    font-size: 11px;
    min-width: 38px;
    text-align: center;
    color: var(--text-secondary);
}

.spinner {
    display: none;
    position: absolute;
    top: 50%; left: 50%;
    transform: translate(-50%, -50%);
    width: 30px; height: 30px;
    border: 3px solid #ccc;
    border-top-color: var(--accent);
    border-radius: 50%;
    animation: spin 1s linear infinite;
    z-index: 10;
}
```

```
@keyframes spin {
    to { transform: translate(-50%, -50%) rotate(360deg); }
}

.hidden {
    display: none !important;
}

/* Eraser UI */
#canvas-result {
    cursor: default;
}

.eraser-active #canvas-result {
    cursor: crosshair;
}

/* Responsive */
@media (max-width: 900px) {
    .workspace {
        flex-direction: column;
    }
    .controls {
        width: 100%;
        flex: none;
    }
    .comparison-view {
        flex-direction: column;
    }
    .canvas-container {
        min-height: 300px;
    }
}

/* Footer */
.app-footer {
    text-align: center;
    padding: 15px 10px;
    margin-top: 15px;
    border-top: 1px solid var(--panel-border);
    font-size: 12px;
    color: var(--text-secondary);
}

.app-footer a {
    color: var(--accent);
```

```
    text-decoration: none;
}

.app-footer a:hover {
    text-decoration: underline;
}

.donate-btn {
    display: inline-block;
    margin-top: 8px;
    padding: 6px 18px;
    background: #e8f5e9;
    color: #2e7d32;
    border: 1px solid #a5d6a7;
    border-radius: 4px;
    font-size: 12px;
    font-family: inherit;
    cursor: pointer;
    text-decoration: none;
    transition: background 0.2s;
}

.donate-btn:hover {
    background: #c8e6c9;
    text-decoration: none;
}
```

فایل: test-math.js

```
const zoomScale = 3.9;
const rect = { left: -481.25, top: -725, width: 1462.5, height: 1950 };
const svgRect = { left: -725, top: -725, width: 1950, height: 1950 };
const cw = 3000, ch = 4000;
const rectWidth = rect.width / zoomScale;
const rectHeight = rect.height / zoomScale;
const canvasRatio = cw / ch;
const rectRatio = rectWidth / rectHeight;
let drawWidth, drawHeight, offsetX, offsetY;
if (canvasRatio > rectRatio) {
    drawWidth = rectWidth;
    drawHeight = rectWidth / canvasRatio;
    offsetX = 0;
    offsetY = (rectHeight - drawHeight) / 2;
} else {
    drawHeight = rectHeight;
```

```
    drawWidth = rectHeight * canvasRatio;
    offsetX = (rectWidth - drawWidth) / 2;
    offsetY = 0;
}
const canvasOffsetX = (rect.left - svgRect.left) / zoomScale;
const canvasOffsetY = (rect.top - svgRect.top) / zoomScale;

function getPoint(imgX, imgY) {
    const svgX = canvasOffsetX + offsetX + (imgX / cw) * drawWidth;
    const svgY = canvasOffsetY + offsetY + (imgY / ch) * drawHeight;
    return {x: svgX, y: svgY};
}

console.log("Point 0,0:", getPoint(0,0));
console.log("Point 3000,4000:", getPoint(3000,4000));
console.log("Point 300,400 (10%):", getPoint(300, 400));
```

test-math2.js فایل:

```
const zoomScale = 3.9;
// Using container dimensions instead of getBoundingClientRect
const container = { clientWidth: 500, clientHeight: 500 };
const canvasRect = { width: 1462.5, height: 1950 }; // scaled rect
const rectWidth = canvasRect.width / zoomScale; // 375
const rectHeight = canvasRect.height / zoomScale; // 500

const canvasOffsetX = (container.clientWidth - rectWidth) / 2;
const canvasOffsetY = (container.clientHeight - rectHeight) / 2;

const cw = 3000, ch = 4000;
const drawWidth = rectWidth;
const drawHeight = rectHeight;

function getPoint(imgX, imgY) {
    const svgX = canvasOffsetX + (imgX / cw) * drawWidth;
    const svgY = canvasOffsetY + (imgY / ch) * drawHeight;
    return {x: svgX, y: svgY};
}

console.log("Point 0,0:", getPoint(0,0));
console.log("Point 3000,4000:", getPoint(3000,4000));
console.log("Point 300,400 (10%):", getPoint(300, 400));
```

فایل test.js:

```
const fs = require('fs');
const jsdom = require("jsdom");
const { JSDOM } = jsdom;
const html = fs.readFileSync('index.html', 'utf8');
const dom = new JSDOM(html, { runScripts: "dangerously" });
const appCode = fs.readFileSync('app.js', 'utf8');
try {
    dom.window.eval(appCode);
    dom.window.document.dispatchEvent(new dom.window.Event('DOMContentLoaded'));
    console.log("No syntax or immediate runtime errors.");
} catch (e) {
    console.error(e);
}
```